



Hybrid Password Authentication Key Exchange in the UC Framework

You Lyu^{1,2}  and Shengli Liu^{1,2} 

¹ Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai 200240, China

{vergil,slliu}@sjtu.edu.cn

² State Key Laboratory of Cryptology, P.O. Box 5159, Beijing 100878, China

Abstract. A hybrid cryptosystem combines two systems that fulfill the same cryptographic functionality, and its security enjoys the security of the harder one. There are many proposals for hybrid public-key encryption (hybrid PKE), hybrid signature (hybrid SIG) and hybrid authenticated key exchange (hybrid AKE). In this paper, we fill the blank of Hybrid Password Authentication Key Exchange (hybrid PAKE).

For constructing hybrid PAKE, we first define an important class of PAKE – *full* DH-type PAKE, from which we abstract sufficient properties to achieve UC security. Our *full* DH-type PAKE framework unifies lots of PAKE schemes like SPAKE2, TBPEKE, (Crs)X-GA-PAKE, and summarizes their common features for UC security.

Stepping from full DH-type PAKE, we propose two generic approaches to hybrid PAKE, *parallel composition* and *serial composition*.

- We propose a generic construction of hybrid PAKE via *parallel composition* and prove that the hybrid PAKE by composing DH-type PAKEs *in parallel* is a full DH-type PAKE and hence achieves UC security, as long as one underlying DH-type PAKE is a full DH-type.
 - We propose a generic construction of hybrid PAKE via *serial composition*, and prove that the hybrid PAKE by composing a DH-type PAKE and another PAKE *in serial* achieves UC security, if *either* the DH-type PAKE is a full DH-type *or* the other PAKE has UC security and the DH-type PAKE only has some statistical properties.
- Our generic constructions of hybrid PAKE result in a variety of hybrid PAKE schemes enjoying different nice features, like round-optimal, high efficiency, or UC security in quantum random oracle model (QROM).

1 Introduction

Password-based cryptosystems are widely deployed in our life, providing authentication and session keys to facilitate secure communications. The task of establishing session keys between two parties in the password only setting is accomplished by Password Authenticated Key Exchange (PAKE) protocols. Since its introduction by Bellare and Meritt [13], many efforts have been devoted to the design and security of PAKE [3, 7, 10, 12, 17, 22, 27, 33].

UC Security of PAKE. There are two primary security notions for PAKE, the game-based security in the Indistinguishability model (IND security) [12] and the simulation-based security under the Universally Composable framework (UC security) [17]. In contrast to the IND model which assumes uniformly distributed passwords, the UC framework permits arbitrary correlations and distributions for passwords and guarantees security amidst composition with arbitrary protocols. Moreover, UC security, even in a relaxed form, can imply IND security as shown in [3]. In practice, PAKE usually serves as a sub-protocol to provide session keys for upper-level protocols, so it always works with other sub-protocols to support flexible cryptographic functionalities. Thanks to UC security of sub-protocols, the whole protocol composed by these sub-protocols still enjoys security in the UC framework. Therefore, UC security is a preferred security notion for PAKE and received lots of concerns.

UC-Secure PAKE from Traditional Assumptions. Numerous efficient UC-secure PAKE protocols have been proposed from traditional number-theoretic assumptions, especially over cyclic groups. Examples include SPEKE [25], SPAKE2 [7], KC-SPAKE2 [33], TBPEKE [31], and CPace [5, 22], etc. These protocols follow Diffie-Hellman style and can be viewed as password-aided Diffie-Hellman key exchange protocol. Notably, most of these protocols are round-optimal [5, 7, 25, 31], i.e., they can be executed in a single simultaneous round, mirroring the round efficiency of Diffie-Hellman protocols. Due to its efficiency and UC security, CPace has been chosen as one of the recommended PAKE protocols by IETF [6].

UC-Secure PAKE from Post-quantum Assumptions. Up to now, there are two approaches to UC-secure PAKE from post-quantum assumptions. One approach resorts to Encrypted Key Exchange (EKE) [13], which takes passwords as the symmetric key and encrypts the transcripts generated by KEM algorithms or None Interactive Key Exchange (NIKE). The UC security is proved by modeling the symmetric encryption/decryption as Ideal Cipher, and hence based on the Ideal Cipher Model (ICM). In fact, the two-round PAKE schemes [8, 11, 32] and the round-optimal PAKE scheme [10] from lattices all follow the EKE approach. However, the EKE approach does not apply to isogenies since there are no instantiations of ideal cipher applying to supersingular elliptic curves yet.

Another approach [30] combines lossy public key encryption (LPKE) and key encapsulation mechanism (KEM), and the resulting PAKE achieves UC security in the Random Oracle model (ROM). If the underlying LPKE is upgraded to extractable LPKE, then the UC security of the PAKE scheme can be proved in Quantum Random Oracle model (QROM). This approach yields three-round PAKE schemes from lattices and three-round PAKE schemes from isogenies.

The EKE approach is more efficient, but the LPKE approach is more general and covers both lattice and isogeny-based instantiations.

We note that the PAKE scheme from lattices in [10] is round-optimal. There do exist round-optimal PAKE schemes [4, 24] from isogenies, but their securities were only proved in IND security model, rather than in UC framework. Hence

round-optimal PAKE with UC security from isogenies is still unknown, which is highlighted in [26]. Naturally, we have the following question:

Q1: Can we find a round-optimal PAKE achieving UC security from isogenies?

Hybrid PAKE from Traditional and Post-quantum Ones. The standardization efforts by NIST are pushing industries to migrate traditional cryptosystems to post-quantum ones. However, the current quantum-resistant hard problems are young and not as well-understood as the traditional number-theoretic hard problems, like factoring, RSA, CDH, DDH etc. Moreover, an imperfect deployment of post-quantum algorithms may compromise the security of the system. So we need more choices for quantum-resistant hard problems and corresponding algorithms to improve robustness. On the other hand, the practical realization of quantum computers remains distant due to challenges like qubit decoherence resulting from the current building materials. So the traditional cryptosystems may retain their vitality for a considerable time. Based on the above observations, many national agencies, like the German BSI [2] and the French ANSSI [1], called for hybrid cryptosystems which combine both traditional and post-quantum ones: the security of hybrid cryptosystems relies on *either* the traditional one *or* the post-quantum one. Meanwhile, hybrid cryptosystems can also work in other ways: it combines two or several traditional ones, or it combines two or several post-quantum ones. The resulting hybrid scheme enjoys the security of the harder of the two or more, improving security robustness.

There have already been lots of works on this topic. Hybrid KEM schemes are proposed in [9, 21], hybrid digital signature schemes in [14, 16], and hybrid authenticated key exchange (AKE) in [19]. However, up to now hybrid PAKE is still missing. In fact, Katz and Rosenberg [26] pointed out: “there is no known hybrid PAKE (in UC framework), let alone a generic method for creating one” and listed the following question as future work:

Q2: Can we find a generic approach to Hybrid PAKE to achieve UC security?

It is not easy to solve Q2. For example, we cannot use EKE-based PAKE schemes [8, 11, 32] from lattices directly in the hybrid PAKE. The reason is as follows. Suppose that the LWE assumption does not hold anymore. Then the encrypted transcript $c = \text{IC.Enc}(pw, \mathbf{A}\mathbf{s} + \mathbf{e})$ in the EKE-based PAKE can help the adversary implement offline dictionary attacks: $\mathcal{A}$ tries every possible choice of password to recover $\mathbf{v} = \text{IC.Dec}(pw, c)$, and $\mathcal{A}$ checks whether $(\mathbf{A}, \mathbf{v})$ is an LWE tuple. If yes, then $\mathcal{A}$ obtains the correct password pw . If this EKE-PAKE is employed by a hybrid PAKE, the hybrid PAKE has already lost its security even if its other PAKE components are UC-secure. This example suggests that hybrid PAKE needs careful and delicate designs.

Our Contribution. We answer the above questions with a full DH-type PAKE Framework and two generic constructions of hybrid PAKE (HPAKE), namely parallel composition and serial composition. See Fig. 1.

- (1) **Full DH-Type PAKE Framework.** We define *full DH-type PAKE* with sufficient properties, which help prove that any full DH-type PAKE has UC

security. We unify many existing PAKE schemes in our full DH-type PAKE framework, including SPAKE2 [7], TBPEKE [31], and (Crs)X-GA-PAKE [4, 24]. Our framework captures the underlying principles of these schemes and explains why they can achieve UC security.

- (2) **Hybrid PAKE via Parallel Composition.** We show how to compose two (or more) PAKEs in parallel to obtain HPAKE. We prove that parallel composition of two (or many) DH-type PAKE yields a full DH-type HPAKE as long as one of the underlying PAKE components is full DH-type. According to (1), the resulting full DH-type HPAKE also achieves UC security which can rely on the security of a single underlying PAKE component.
- (3) **Hybrid PAKE via Serial Composition.** We show how to compose two PAKEs in serial to obtain Hybrid PAKE. We prove that the Hybrid PAKE achieves UC security, if either the first PAKE is a full DH-type one or the second PAKE has UC security (and the DH-type only has some statistical properties). Moreover, if the second PAKE has UC security in QROM, then the resulting Hybrid PAKE also enjoys UC security in QROM.

Our result (1) suggests that the (Crs)X-GA-PAKE schemes [4, 24] are round-optimal UC-secure PAKEs from isogenies, and hence solves the aforementioned question *Q1*. Our results (2) and (3) solve the aforementioned question *Q2*.

According to Fig. 1, our result implies abundant hybrid PAKE schemes. Below we list a few.

- **Round-optimal Hybrid PAKE.** Parallel composition of SPAKE2 [7] and (Crs)X-GA-PAKE [4, 24] or parallel composition of SPAKE2, (Crs)X-GA-PAKE and TBPEKE [31] yields a round-optimal hybrid PAKE scheme relying on either the traditional assumptions or post-quantum assumption from isogenies in ROM.
- **Efficient Hybrid PAKE.** Serial composition of SPAKE2 and the recent EKE-based PAKE CHIC [8] yields an efficient hybrid PAKE scheme relying on either the traditional assumptions or post-quantum assumption from lattices in ROM. Recall that EKE-based PAKEs are the most efficient ones among the existing UC-secure PAKEs from post-quantum assumptions. This hybrid PAKE is efficient since it inherits efficiency from SPAKE2 and CHIC.
- **Hybrid PAKE in QROM.** Serial composition of SPAKE2 and PAKE^{QRO} [30] yields a hybrid PAKE scheme whose UC-security is achieved in QROM as long as PAKE^{QRO} has UC security in QROM.

Technique Overview. UC security considers the indistinguishability between the real world and the ideal world for an environment $\mathcal{Z}$. In the real world, $\mathcal{Z}$ initiates passwords for the parties, sees the interactions between parties and the adversary $\mathcal{A}$, and obtains the session key output by parties. In the ideal world, the simulator Sim simulates the view of $\mathcal{Z}$ without pw by accessing an ideal functionality $\mathcal{F}$ for PAKE. For UC security, we have to design PAKE and a corresponding simulator Sim to accomplish the following tasks:

Task I. Sim can simulate all the interaction transcripts for both passive attacks and active attacks without passwords.

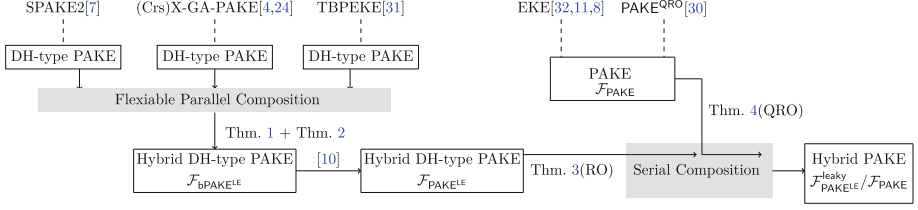


Fig. 1. Schematic overview of constructing hybrid PAKEs via parallel composition and serial composition. Here “*flexible* parallel composition” means that the DH-type PAKE components can be flexibly chosen: one, two or many. “----” denotes instantiations. $\boxed{A} \rightarrow \boxed{B} \rightarrow \boxed{C}$ denotes that A is used for C via B.

Task II. PAKE has pseudo-random session keys in case of passive attacks and unsuccessful active attacks (i.e., attacks with wrong passwords), so Sim can sample random keys for simulation, which is done by invoking $\mathcal{F}$.

Task III. Sim can extract password in case of active attacks, and keep the consistency of session keys between $\mathcal{Z}$ and $\mathcal{A}$ for successful active attacks (i.e., attacks with correct passwords) with the help of access to Testpw (one access per protocol instance) provided by $\mathcal{F}$.

Next we give a high-level description of our full DH-type PAKE framework and explain why it can achieve UC security. Then we show how to combine DH-type PAKEs via parallel composition and how to combine a DH-type PAKE with another PAKE via serial composition to construct hybrid PAKEs.

FULL DH-TYPE PAKE FRAMEWORK. The starting point is a DH-type PAKE (see Fig. 6). The two parties can invoke algorithms $\text{DHInit}(pw)$ and $\text{DHResp}(pw)$ to generate round messages m_1 and m_2 (and their state st_1 and st_2), and compute their DH-key w by $\text{Comp}_i(pw, st_1, m_1, m_2)$ and $\text{Comp}_r(pw, st_2, m_1, m_2)$, respectively. The session key is a hash value $\text{Key} = H(pw, m_1, m_2, w)$. To support UC security, we equip DH-type PAKE with simulation algorithms SimInit , SimResp , SimComp_i , and SimComp_r satisfying the following properties to help construct a simulator Sim . Here we require SimComp_i and SimComp_r are deterministic algorithms.

Perfect Simulation of Round Messages. SimInit and SimResp should simulate messages m_1 and m_2 perfectly without any password, and hence Sim can invoke them to accomplish task I.

One-Wayness of the DH-Key in case of Passive Attacks. It is hard for $\mathcal{A}$ to compute w given (m_1, m_2) , so session key $\text{Key} = H(pw, m_1, m_2, w)$ is uniform in the real world due to the one-wayness of w and random oracle H . So Sim can sample a random key to simulate Key . This accomplishes task II.

Unique Password Extraction in case of Active Attacks. To support this, we have the following two requirements.

- **Perfect Simulation of DH-key.** For $\mathcal{A}$ ’s active attack with $\tilde{m}_1$, the DH-key $w := \text{SimComp}_i(td, pw, st_1, \tilde{m}_1, m_2)$, has identical distribution as the output of Comp_i . A similar requirement applies to SimComp_r as well.

- **Unique password for Initiator/Responder.** It is hard for $\mathcal{A}$ to present two pairs (pw, w) and (pw', w') with $pw \neq pw'$ such that $w = \text{SimComp}_i(td, pw, st_1, \tilde{m}_1, m_2)$ and $w' = \text{SimComp}_i(td, pw', st_1, \tilde{m}_1, m_2)$. A similar requirement applies to SimComp_r as well.

For an active attack, Sim can extract at most one pair (pw, w) s.t. $w = \text{SimComp}_i(td, pw, st_1, \tilde{m}_1, m_2)$ (or $w = \text{SimComp}_r(td, pw, st_2, m_1, \tilde{m}_2)$) from $\mathcal{A}$'s hash queries $H(pw, m_1, m_2, w)$. $\text{Testpw}(pw)$ helps Sim to determine whether pw is correct or not. Therefore, in case of unsuccessful active attacks (pw is not correct), $\mathcal{A}$ did not query $H(pw^*, m_1, m_2, w)$ with the correct password pw^* due to the unique password property, so the session key $\text{sKey} := H(pw^*, m_1, m_2, w)$ is uniform in the real world, which accomplishes task II. For successful active attacks, Sim can extract the correct password and honestly set the session key as $\text{sKey} := H(pw^*, m_1, m_2, w)$ to keep consistency, which accomplishes task III.

We call the DH-type PAKE satisfying the above properties a *full* DH-type PAKE. The above analysis shows that full DH-type PAKE has UC security¹ and enjoys the round-optimal property². Actually, SPAKE2 [7], TBPEKE [31], and (Crs)X-GA-PAKE [4, 24] all fall into our full DH-type PAKE framework.

The full DH-type PAKE does not end with UC security. In fact, the good property of “*perfect simulation of round messages and DH-keys*” implies that the round messages contain no information about the password at all, and the DH-key in the real world can be simulated perfectly in case of successful active attacks. Therefore, DH-type PAKEs with this property can serve as vital components in constructing hybrid PAKE no matter via parallel or serial compositions.

HYBRID PAKE VIA PARALLEL COMPOSITION. Given two PAKE schemes PAKE_1 and PAKE_2 , hybrid PAKE via parallel composition means that two parties $\mathcal{P}_i$ and $\mathcal{P}_j$ run PAKE_1 and PAKE_2 with password pw in parallel to obtain their session keys k_1 and k_2 respectively, and compute the session key $\text{sKey} := \text{Hash}(pw, k_1, k_2, \text{trans})$. See the left part of Fig. 2.

If PAKE_1 and PAKE_2 are both DH-type PAKE, then the hybrid PAKE sets its DH-key as $w := (w^{(1)}, w^{(2)})$ and its session key as $\text{sKey} := H(pw, w^{(1)}, w^{(2)}, \text{trans})$, where $w^{(1)}$ (resp. $w^{(2)}$) is the DH-key of PAKE_1 (resp. PAKE_2).

If one PAKE scheme, say PAKE_1 , is a full DH-type PAKE, and the other PAKE scheme, say PAKE_2 , is DH-type PAKE with the aforementioned good property, then we show that the hybrid PAKE via parallel composition is a full DH-type PAKE. The simulation algorithms of hybrid PAKE are parallel invocations of the underlying simulation algorithms of PAKE_1 and PAKE_2 . For

¹ Note that the adversary can make the hash query after session key generation. Accordingly, Sim extracts the password after session key generation. This UC security is reflected by lazy-extraction of the ideal functionality $\mathcal{F}_{\text{PAKELE}}$ (see Fig. 4).

² Due to the independence of the two round messages, full DH-type PAKE is actually a bare PAKE [10], which can emulate $\mathcal{F}_{\text{bPAKELE}}$, better than $\mathcal{F}_{\text{PAKELE}}$. See Sect. 3.2 for details.

example, $\text{SimComp}_i := (\text{SimComp}_i^{(1)}, \text{SimComp}_i^{(2)})$, where $\text{SimComp}_i^{(1)}$ belongs to PAKE_1 and $\text{SimComp}_i^{(2)}$ belongs to PAKE_2 .

- The good property of perfect simulation of round messages and DH-key for hybrid PAKE inherits from PAKE_1 and PAKE_2 .
- The one-wayness of $w = (w^{(1)}, w^{(2)})$ follows from that of $w^{(1)}$.
- PAKE_1 's property of unique password for initiator implies there exists at most one $(pw, w^{(1)})$ s.t. $w^{(1)} = \text{SimComp}_i^{(1)}(td^{(1)}, pw, st_1^{(1)}, m_1^{(1)}, m_2^{(1)})$. Meanwhile, the unique pw determines the unique $w^{(2)} := \text{SimComp}_i^{(2)}(td^{(2)}, pw, st_1^{(2)}, m_1^{(2)}, m_2^{(2)})$. Consequently, there exists at most one $(pw, w = (w^{(1)}, w^{(2)}))$ satisfying the above two equations, and hybrid PAKE also has the property of unique password for initiator. A similar argument applies to the responder.

Therefore, the hybrid PAKE via parallel composition falls into the full DH-type PAKE framework and hence achieves UC security as long as one component is a full DH-type one and the other has the good property of “perfect simulation of round messages and DH-keys”. This result can be extended to parallel composition of *multiple* DH-type PAKEs.

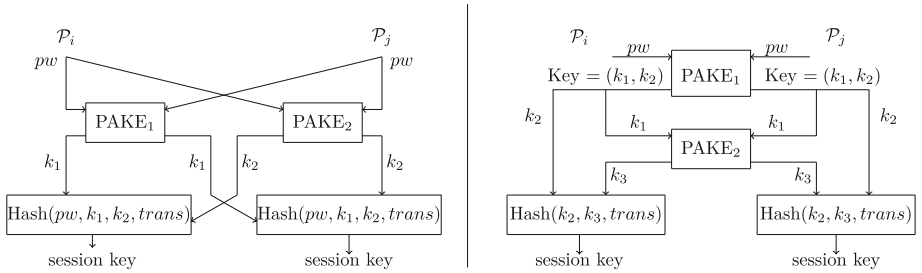


Fig. 2. Parallel composition (left) & serial composition (right) for hybrid PAKE.

HYBRID PAKE VIA SERIAL COMPOSITION. Given two PAKE schemes PAKE_1 and PAKE_2 , we propose how to construct hybrid PAKE via serial composition. Two parties P_i and P_j first run PAKE_1 with password pw to share PAKE_1 's session key $\text{Key} = (k_1, k_2)$, then run PAKE_2 with password k_1 to share PAKE_2 's session key k_3 . Finally they set the hybrid PAKE's session key as $\text{sKey} := \text{Hash}(k_2, k_3, \text{trans})$. See the right part of Fig. 2.

To show UC security of this hybrid PAKE, we will justify the following two statements.

1. If PAKE_1 is a full DH-type PAKE, then we show that the hybrid PAKE via serial composition has UC security.
2. If PAKE_2 scheme has UC security in (Q)ROM and PAKE_1 is a DH-type PAKE with the good property of “perfect simulation of round messages and DH-keys”, then we show that the hybrid PAKE via serial composition has UC security in (Q)ROM.

If PAKE_1 scheme is a full DH-type PAKE, then it is UC-secure and associated with a simulator Sim_1 . Next we prove the validity of statement 1.

- Thanks to the UC security of PAKE_1 , Sim_1 can help Sim simulate the PAKE_1 's transcript and its session key $\text{Key} = (k_1, k_2)$. Then Sim can invoke PAKE_2 with password k_1 to generate PAKE_2 's transcript and its session key k_3 . Lastly, Sim simulates hybrid PAKE's session key with $\text{sKey} := \text{Hash}(k_2, k_3, \text{trans})$. Clearly, the transcript simulation is perfect, and hence task I is accomplished.
- In case of passive attacks or unsuccessful active attacks (with wrong pw), $\text{Key} = (k_1, k_2)$ of PAKE_1 is uniform according to UC security of PAKE_1 . Consequently, $\text{sKey} = \text{Hash}(k_2, k_3, \text{trans})$ is uniform as well due to the uniformity of k_2 , and hence task II is accomplished.
- In case of successful active attacks (with correct pw), Sim_1 can help Sim to extract the correct password, and keep the consistency of $\text{Key} = (k_1, k_2)$, thanks to PAKE_1 's UC security again. Since k_3 is derived from PAKE_2 with password k_1 , the input (k_2, k_3, trans) are consistent, so the session key $\text{sKey} = \text{Hash}(k_2, k_3, \text{trans})$ keeps consistency to $\mathcal{Z}$ and $\mathcal{A}$, and hence task III is accomplished. Note that Sim_1 can help Sim to extract pw in case of active attacks, so Sim can tell the correctness of pw with the help of $\text{Testpw}(pw)$.

For statement 2, we suppose that PAKE_2 has UC security in (Q)ROM and is associated with a simulator Sim_2 , and we also suppose that PAKE_1 is a DH-type PAKE with the good property of “perfect simulation of round messages and DH-keys” with the four simulation algorithms SimInit , SimResp , SimComp_i , SimComp_r . Now we construct simulator Sim for hybrid PAKE as follows.

- Sim invokes SimInit and SimResp to generate transcript of PAKE_1 , and invokes Sim_2 to generate transcript and session key k_3 of PAKE_2 . Due to the good property of PAKE_1 and the UC security of PAKE_2 , Sim gives an indistinguishable transcript of hybrid PAKE, and hence task I is accomplished.
- In case of passive attacks, the UC security of PAKE_2 guarantees the uniformity of k_3 , which further implies the uniformity of $\text{sKey} = \text{Hash}(k_2, k_3, \text{trans})$, and hence task II for passive attacks is accomplished.
- Upon an active attack, due to the UC security of PAKE_2 , Sim_2 can extract the unique password guessing k_1 of PAKE_2 . Then Sim searches the hash list to find (pw, m_1, m_2, w) s.t. $(k_1, k_2) = H(pw, m_1, m_2, w)$, where (m_1, m_2) is the transcript of PAKE_1 and w is the DH-key of PAKE_1 (i.e., $w = \text{SimComp}_i(td, pw, st_1, m_1, m_2)$ or $w = \text{SimComp}_r(td, pw, st_2, m_1, m_2)$). In this way, Sim extracts a unique guessing password pw from $\mathcal{A}$'s attacks. Then Sim resorts to $\text{Testpw}(pw)$ to check the correctness of the password.
 - In case of unsuccessful active attacks (pw is not correct), adversary $\mathcal{A}$ guesses a wrong password, which leads to a wrong k_1 for PAKE_2 . This implies an unsuccessful active attack for PAKE_2 . In this case, the UC security of PAKE_2 guarantees the uniformity of k_3 , which further implies the uniformity of $\text{sKey} = \text{Hash}(k_2, k_3, \text{trans})$. Therefore, task II for unsuccessful active attacks is accomplished.

- In case of successful active attacks (pw is correct and hence $\text{Key} = (k_1, k_2)$ can be correctly derived from pw by Sim), Sim_2 can keep the consistency of PAKE_2 's session key k_3 due to PAKE_2 's UC security. Consequently, Sim can set session key as $\text{sKey} := \text{Hash}(k_2, k_3, \text{trans})$ to keep consistency between $\mathcal{A}$ and $\mathcal{Z}$, and hence task III is accomplished.

Note that Sim needs to extract password pw from k_1 , and this extraction is realized by formalizing H as a random oracle. By resorting to the online-extractable technique [20], the extraction can also be realized in QROM where H is formalized as a quantum random oracle. If PAKE_2 has UC security in QROM, then the resulting hybrid PAKE has UC security in QROM as well.

We only give a high-level description of our generic construction of hybrid PAKE and their analysis for UC security. In fact, there are many subtleties to deal with in the formal constructions and security proofs. We refer to Sects. 3 and 4 for details.

2 Preliminary

If x is defined by y or y is assigned to x , we write $x := y$. For $\mu \in \mathbb{N}$, define $[\mu] := \{1, 2, \dots, \mu\}$. Denote by $x \leftarrow_s \mathcal{X}$ the procedure of sampling x from set $\mathcal{X}$ uniformly at random. All algorithms $y \leftarrow \mathcal{A}(x)$ are probabilistic, take the security parameter 1^λ as implicit input unless stated otherwise. $X \equiv Y$ denotes that X and Y are independent identically distributed. $[a \stackrel{?}{=} b]$ checks whether $a = b$ or not, and it returns 1 if $a = b$ and returns 0 otherwise. Retrieving records $([a], b, [c], d)$ means records are retrieved according to b and d . ROM and QROM are discussed in our full version [29].

2.1 Password-Based Authenticated Key Exchange

A (two-round) PAKE scheme consists of four algorithms:

- **Setup** : It outputs a public parameter pp . All the remaining algorithms take pp as input, and we omit it for simplicity.
- **Init**(pw) : It takes as input a password pw , and outputs a first-round message m_1 and a round state st .
- **Resp**(pw, m_1) : It takes as input a password pw and the first-round message m_1 , and outputs a second-round message m_2 and a session key Key .
- **Deri**(pw, st, m_2) : It takes as input a password pw , a round state st and the second-round message m_2 , and outputs a session key Key .

2.2 The Universal Composable Framework

We present a concise overview of the UC framework. Figure 3 shows the picture of the “real world” execution of a protocol Π and the “ideal world” execution with a simulator Sim . The environment $\mathcal{Z}$ represents higher-level protocols invoking Π as a sub-protocol. The adversary $\mathcal{A}$ models a completely insecure network and

continuously interacts with $\mathcal{Z}$. Each party instance sends and receives messages via $\mathcal{A}$, and $\mathcal{A}$ can do dropping, injecting, and modifying messages at will.

In the real world, parties execute protocol instances as described in Π . They receive their inputs (e.g. passwords, session identifier, role) from $\mathcal{Z}$ and send their outputs (e.g. session key, sub-session identifier) to $\mathcal{Z}$.

In the ideal world, parties become “dummy” entities, passing inputs directly from $\mathcal{Z}$ to an ideal functionality $\mathcal{F}$, and outputs directly from $\mathcal{F}$ to $\mathcal{Z}$. The simulator Sim interacts with $\mathcal{Z}$ and $\mathcal{F}$. Sim must indistinguishably simulate network transcripts as in the real world and provide appropriate inputs to $\mathcal{F}$ to generate outputs indistinguishable from those produced by real-world instances.

In a nutshell, we say protocol Π securely emulates ideal functionality $\mathcal{F}$ if for any efficient adversary $\mathcal{A}$, there exists an efficient simulator Sim such that no efficient environment $\mathcal{Z}$ can distinguish between the actual protocol executions in the real world and the hypothetical executions simulated in the ideal world.

If protocol Π is constructed from some subprotocol Π' , which securely emulates an ideal functionality $\mathcal{G}$, then the description of Π and its UC security may be proven in the $\mathcal{G}$ -hybrid model. In the $\mathcal{G}$ -hybrid model, Π' is replaced by $\mathcal{G}$, and parties and the adversary interact with $\mathcal{G}$ according to its specification. As for the UC security, Sim must additionally simulate the ideal functionality of $\mathcal{G}$.

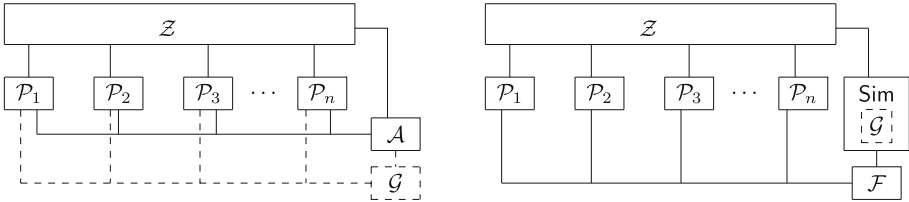


Fig. 3. Left: the real world $\text{Real}_{\mathcal{Z},\mathcal{A}}$ (without dashed lines and dashbox), and the real world $\text{Real}^{\mathcal{G}}_{\mathcal{Z},\mathcal{A}}$ in $\mathcal{G}$ -hybrid model (with dashed lines and dashbox). Right: the ideal world $\text{Ideal}_{\mathcal{Z},\text{Sim}}$ (without dashbox) and the ideal world $\text{Ideal}_{\mathcal{Z},\text{Sim}[\mathcal{G}]}$ in $\mathcal{G}$ -hybrid model (with dashbox).

2.3 PAKE in UC Framework

The original ideal functionality for PAKE was presented in [17]. Later it was extended to a multi-session version $\mathcal{F}_{\text{PAKE}}$ in [18] and a lazy-extraction version $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$ in [3]. The two extended versions are shown in Fig. 4.

Unlike the single-session PAKE functionality in [17], the multi-session extension $\mathcal{F}_{\text{PAKE}}$ mandates that each individual protocol instance possesses a globally unique sub-session identifier $ssid$. The session identifier sid becomes a global parameter which all parties in the protocol can rely on. See Fig. 4.

The lazy-extraction version $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$ was proposed by Abdalla et al. [3], and it permits adversary $\mathcal{A}$ to guess a party’s password pw (at most once) even

after the session key is generated. If the guess is correct, $\mathcal{A}$ acquires the party's session key. Otherwise, $\mathcal{A}$ obtains only a random key. In comparison, in the original definition of $\mathcal{F}_{\text{PAKE}}$ in [17], $\mathcal{F}_{\text{PAKE}}$ does not reply $\mathcal{A}$ at all when $\mathcal{A}$ issues a late password guess after the session key generation. Notably, in the definition of $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$ in [3], the adversary is not explicitly informed whether its password guess is correct. In this paper we also introduce a leaky variant of $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$ that notifies the adversary of the password guess's outcome. UC security with $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$ and its leaky variant may be not as strong as that with $\mathcal{F}_{\text{PAKE}}$, but leaking bits of success or failure and late test after session key generation do reflect the features of the real world. Hence ideal functionality of $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$ is a reasonable one and adopted in many works [3, 5, 10, 28]. See Fig. 4.

Session Initialization

Upon receiving a query (NewSession, $sid, \mathcal{P}, ssid, \mathcal{CP}, pw$, role) **from** $\mathcal{P}$:
 Send (NewSession, $sid, \mathcal{P}, ssid, \mathcal{CP}$, role) to $\mathcal{A}$
 If this is the first NewSession query on $ssid$,
 or this is the second NewSession query on $ssid$ and record $(\mathcal{CP}, ssid, \mathcal{P}, pw')$ exists:
 Record $(\mathcal{P}, ssid, \mathcal{CP}, pw)$ and mark it **fresh**

Active Session Attacks

Upon receiving a query (Testpw, $sid, \mathcal{P}, ssid, pw'$) **from** $\mathcal{A}$:
 If there is a **fresh** record $(\mathcal{P}, ssid, [\mathcal{CP}, pw])$:
 If $pw' = pw$: mark the record **compromised** and reply with “correct guess”
 If $pw' \neq pw$: mark the record **interrupted** and reply with “wrong guess”

Lazy Password Extraction

Upon receiving a query (RegisterTest, $sid, \mathcal{P}, ssid$) **from** $\mathcal{A}$:
 If there is a **fresh** record $(\mathcal{P}, ssid, [\mathcal{CP}, pw])$:
 mark the record **interrupted** and add a “tested” flag.
Upon receiving a query (LateTestPwd, $sid, \mathcal{P}, ssid, pw'$) **from** $\mathcal{A}$:
 If there is a **completed** record $(\mathcal{P}, ssid, [\mathcal{CP}, pw])$ with flag **tested**:
 retrieve the record $(\mathcal{P}, ssid, \mathcal{CP}, pw, \text{Key})$, remove the flag **tested**
 If $pw = pw'$: return Key and “correct guess” to $\mathcal{A}$
 Else: sample a random $\text{Key}' \leftarrow \mathcal{K}$, return Key' and “wrong guess” to $\mathcal{A}$

Key Generation

Upon receiving a query (NewKey, $sid, \mathcal{P}, ssid, \text{Key}^*$) **from** $\mathcal{A}$:
 If there is a record $(\mathcal{P}, ssid, [\mathcal{CP}, pw])$ not marked **completed**:
 If the record is **compromised**: set $\text{Key} := \text{Key}^*$
 If the record is **fresh**, and there exists a **completed** record $(\mathcal{CP}, ssid, \mathcal{P}, pw)$ which was **fresh**
 when $\mathcal{CP}$ output $(sid, ssid, \text{Key}')$: set $\text{Key} := \text{Key}'$
 In all other cases, sample a random Key
 Finally, mark the record $(\mathcal{P}, ssid, \mathcal{CP}, pw)$ **completed** and store $(\mathcal{P}, ssid, \mathcal{CP}, pw, \text{Key})$

Fig. 4. The ideal functionality $\mathcal{F}_{\text{PAKE}}$ for PAKE. Adding the gray part to but deleting the [part] from $\mathcal{F}_{\text{PAKE}}$ defines a lazy extraction version called $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$. Adding both gray and [part] to $\mathcal{F}_{\text{PAKE}}$ defines a leaky version named $\mathcal{F}_{\text{PAKE}^{\text{LE}}}^{\text{leaky}}$.

Very recently, Barbosa et al. proposed the so-called *Bare PAKE functionality* $\mathcal{F}_{\text{bPAKE}}$ and its lazy-extraction variant $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$ in [10]. We recall $\mathcal{F}_{\text{bPAKE}}$ and $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$ in Fig. 5. $\mathcal{F}_{\text{bPAKE}}$ (and $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$) mainly offers two advantages over the original PAKE functionality $\mathcal{F}_{\text{PAKE}}$.

- $\mathcal{F}_{\text{bPAKE}}$ eliminates the need for pre-shared sub-session identifiers $ssid$ and the unique peer identifiers cid indicating counter party $\mathcal{CP}$. Each party's input to $\mathcal{F}_{\text{bPAKE}}$ includes the global parameter sid , a locally unique instance identifier i , the party's identifier id , and a password pw . The sub-session identifier and peer identifier are generated as outputs of $\mathcal{F}_{\text{bPAKE}}$. This simplification enhances practicality and reduces implementation hurdles.
- $\mathcal{F}_{\text{bPAKE}}$ enables parties to reuse their session states. An instance can utilize its existing session state to communicate with multiple other instances. $\mathcal{F}_{\text{bPAKE}}$ guarantees that reusing a single instance's state is equivalent to running multiple independent PAKE instances with the same password.

In $\mathcal{F}_{\text{bPAKE}}$ (see Fig. 5), **PassiveNewKey** captures $\mathcal{A}$'s passive attacks of matching instances and corresponds to the **NewKey** query to $\mathcal{F}_{\text{PAKE}}$ with a passive attack. **ActiveNewKey** captures $\mathcal{A}$'s active attacks and corresponds to the combination of **Testpw** and **NewKey** queries to $\mathcal{F}_{\text{PAKE}}$. For example, to attack an instance $(sid, \mathcal{P}, i)$, $\mathcal{A}$ can query $\mathcal{F}_{\text{bPAKE}}$ with **(ActiveNewKey, $sid, \mathcal{P}, i, pw', \text{Key}', ssid, cpid$)**. If $pw' = pw$, then $\mathcal{F}_{\text{bPAKE}}$ sets the session key as Key' and sets the peer identifier of instance $(sid, \mathcal{P}, i)$ as cid . Otherwise, choose $\text{Key} \leftarrow_s \mathcal{K}$ as the session key.

Remark 1. Our UC frameworks for PAKE do not consider (adaptive) corruptions with which the adversary may obtain the internal state of a running protocol instance. But we stress that our PAKE supports adaptive password corruptions since the environment is in charge of passwords in the UC model.

Remark 2. In our security proof, we assume the partnered parties always run the protocol with the correct password. The case of inputting incorrect password can be easily modelled by the UC framework in [33], which additionally captures the 1-bit leakage of whether the party inputs the correct password. When the session key of PAKE is used for the latter authentication, this bit will be leaked anyway. We remark that the work in [23] consider a stronger model which allows incorrect password input but without any leakage.

3 Full DH-Type PAKE: Definition, UC Security, and Parallel Composition

We introduce the concept of full DH-type PAKE in Subsec. 3.1, and then we prove that any full DH-type PAKE scheme achieves the bare PAKE functionality $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$ in the UC framework in Subsec. 3.2. Finally, we show how to construct hybrid DH-type PAKE schemes by combining multiple DH-type PAKE protocols in parallel in Subsec. 3.3.

3.1 Definition of Full DH-Type PAKE

A Diffie-Hellman type (DH-type) PAKE protocol is a two-round PAKE and captured by five algorithms: **DHSetup** sets up the system; An initiator uses

The functionality is parameterized by a set S (initialized to be empty) and a session key space $\mathcal{K}$.

Session Initialization

Upon receiving a query (NewSession, $sid, i, pw, id, \text{role} \in \{\text{Initiator}, \text{Responder}\}$) **from** $\mathcal{P}$:

Send (NewSession, $sid, \mathcal{P}, i, id, \text{role}$) to $\mathcal{A}$

If there is no record $(\mathcal{P}, i, [pw, id])$: record $(\mathcal{P}, i, [pw, id])$

Key Generation

Upon receiving a query (ActiveNewKey, $sid, \mathcal{P}, i, pw', \text{Key}', ssid, cpid$) **from** $\mathcal{A}$:

If there is a record $(\mathcal{P}, i, [pw, id])$:

If there is a record $(\text{ses}_{act}, \mathcal{P}, i, ssid, cpid, [\text{Key}])$: output $(sid, i, \text{Key}, ssid, cpid)$ to $\mathcal{P}$

Else, if $ssid \notin S$:

Add $ssid$ to S

If $pw' = pw$: set $\text{Key} := \text{Key}'$; Else set $\text{Key} \leftarrow_s \mathcal{K}$

If $pw' = \perp$: record $(\text{latetest}, \mathcal{P}, i, ssid, pw, \text{Key} \leftarrow_s \mathcal{K})$

Record $(\text{ses}_{act}, \mathcal{P}, i, ssid, cpid, \text{Key})$

Output $(sid, i, \text{Key}, ssid, cpid)$ to $\mathcal{P}$

Upon receiving a query (PassiveNewKey, $sid, \mathcal{P}, i, \mathcal{CP}, i', ssid$) **from** $\mathcal{A}$:

If there is a record $(\mathcal{P}, i, [pw, id])$:

If there is a record $(\text{ses}_{hbc}, \mathcal{P}, i, [\mathcal{CP}, i'], ssid, [cpid, \text{Key}])$: output $(sid, i, \text{Key}, ssid, cpid)$ to $\mathcal{P}$

If there is a record $(\mathcal{CP}, i', [pw', id'])$: set $cpid := id'$ and do:

If $ssid \notin S$:

Add $ssid$ to S

$\text{Key} \leftarrow_s \mathcal{K}$, record $(\text{ses}_{hbc}, \mathcal{P}, i, \mathcal{CP}, i', ssid, cpid, \text{Key})$

Output $(sid, i, \text{Key}, ssid, cpid)$ to $\mathcal{P}$

Else if there is a record $(\text{ses}_{hbc}, \mathcal{CP}, i', \mathcal{P}, i, ssid, id, [\text{Key}'])$:

If $pw' = pw$: set $\text{Key} := \text{Key}'$; Else $\text{Key} \leftarrow_s \mathcal{K}$

Save $(\text{ses}_{hbc}, \mathcal{P}, i, \perp, \perp, ssid, cpid, \text{Key})$

Output $(sid, i, \text{Key}, ssid, cpid)$ to $\mathcal{P}$

Late Password Test Attack

Upon receiving a query (LateTestPwd, $sid, \mathcal{P}, i, ssid, pw'$) **from** $\mathcal{A}$:

If there is a record $(\text{latetest}, \mathcal{P}, i, ssid, [pw, \text{Key}])$:

Delete the record

If $pw = pw'$ then set $\text{Key}' := \text{Key}$; else sample $\text{Key}' \leftarrow_s \mathcal{K}$

Output Key' to $\mathcal{A}$

Fig. 5. The ideal functionality $\mathcal{F}_{\text{bPAKE}}$ for bare PAKE. Adding gray part to $\mathcal{F}_{\text{bPAKE}}$ defines a lazy extraction version of PAKE called $\mathcal{F}_{\text{bPAKELE}}$.

DHInit(pw) to generate the first-round message; A responder uses DHResp(pw) to generate the second-round message; The initiator uses Comp_i to generate the DH-key w ; The responder uses Comp_r to generate the DH-key w ; Finally, both the initiator and responder take the password pw , the DH-key w , and the transcript of the protocol as the input of a hash function to derive the session key Key from the hash function. The DH-type is featured by the fact that the generation of second-round message does not rely on the first-round message (and hence DH-type PAKE can be executed in a single simultaneous round). See Fig. 6 for details.

Definition 1. A DH-type PAKE protocol is a two-round protocol with the following five algorithms.

- DHSetup: It outputs a public parameter pp and a trapdoor td . All the remaining algorithms take pp as input, and we omit it for simplicity.
- DHInit(pw): It takes as input a password pw , and outputs a message m_1 and a secret state st_1 .

- $\text{DHResp}(pw)$: It takes as input a password pw , and outputs a message m_2 and a secret state st_2 .
- $\text{Comp}_i(pw, st_1, m_1, m_2)$: It is a deterministic algorithm, which takes as input a password pw , a secret state st_1 , and two messages m_1, m_2 , and outputs a DH-key w .
- $\text{Comp}_r(pw, st_2, m_1, m_2)$: It is a deterministic algorithm, which takes as input a password pw , a secret state st_2 , and two messages m_1, m_2 , and outputs a DH-key w .

Correctness. For all pw and $(pp, td) \leftarrow \text{DHSetup}$, it holds that

$$\Pr \left[\begin{array}{l} (m_1, st_1) \leftarrow \text{DHInit}(pw) \\ (m_2, st_2) \leftarrow \text{DHResp}(pw) \end{array} : \text{Comp}_i(pw, st_1, m_1, m_2) = \text{Comp}_r(pw, st_2, m_1, m_2) \right] = 1.$$

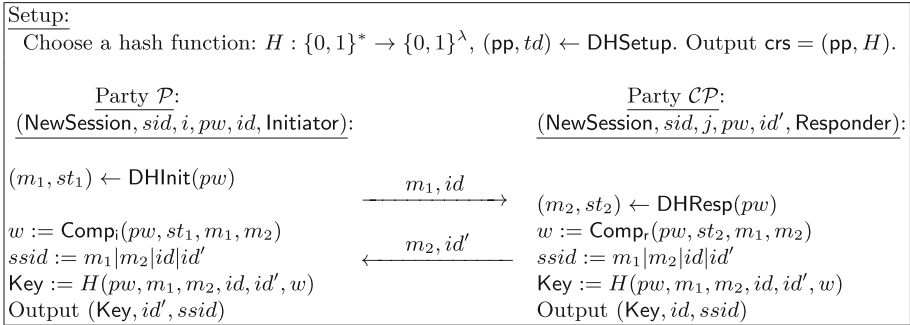


Fig. 6. DH-type PAKE protocol.

Definition 2. (Full DH-type PAKE). A two-round PAKE protocol is a full DH-type PAKE, if it is a DH-type one (DHSetup , DHInit , DHResp , Comp_i , Comp_r) associated with simulation algorithms (SimInit , SimResp , SimComp_i , SimComp_r) and satisfies five properties. The simulation algorithms are defined as follows.

- $\text{SimInit}(pp)$: It takes as input a public parameter pp , and outputs a message m_1 and a secret state st_1 .
- $\text{SimResp}(pp)$: It takes as input a public parameter pp , and outputs a message m_2 and a secret state st_2 .
- $\text{SimComp}_i(td, pw, st_1, m_1, m_2)$: It is a deterministic algorithm, which takes as input a trapdoor td , a password pw , a state st_1 , and two messages m_1, m_2 and outputs a key w .
- $\text{SimComp}_r(td, pw, st_2, m_1, m_2)$: It is a deterministic algorithm, which takes as input a trapdoor td , a password pw , a state st_2 , and two messages m_1, m_2 and outputs a key w .

There are five properties defined for full DH-type PAKE protocols.

- ① **One-Wayness of the DH-Key:** The advantage function $\text{Adv}_{\text{key}}^{\text{ow}}(\mathcal{A})$ is negligible for all PPT adversary $\mathcal{A}$ access to oracles $\mathcal{O}_1(m_1^*, st_1^*, \cdot, \cdot, \cdot)$ and $\mathcal{O}_2(m_2^*, st_2^*, \cdot, \cdot, \cdot)$, where $\text{Adv}_{\text{key}}^{\text{ow}}(\mathcal{A}) :=$

$$\Pr \left[\begin{array}{l} (\text{pp}, td) \leftarrow \text{DHSetup}, pw \leftarrow \mathcal{A}(\text{pp}) \\ (m_1^*, st_1^*) \leftarrow \text{DHInit}(pw) \\ (m_2^*, st_2^*) \leftarrow \text{DHResp}(pw) \\ w \leftarrow \mathcal{A}^{\mathcal{O}_1(m_1^*, st_1^*, \cdot, \cdot, \cdot), \mathcal{O}_2(m_2^*, st_2^*, \cdot, \cdot, \cdot)}(\text{pp}, pw, m_1^*, m_2^*) \end{array} : w = \text{Comp}_i(pw, st_1^*, m_1^*, m_2^*) \right],$$

$\mathcal{O}_1(m_1^*, st_1^*, pw', m_2', w') := [w' \stackrel{?}{=} \text{Comp}_i(pw', st_1^*, m_1^*, m_2')] \text{ and } \mathcal{O}_2(m_2^*, st_2^*, pw', m_1', w') := [w' \stackrel{?}{=} \text{Comp}_r(pw', st_2^*, m_1', m_2^*)].$

- ② **Perfect Simulation for Initiator:** For all $(\text{pp}, td) \leftarrow \text{DHSetup}$, all pw , and all m_2 , it holds that

$$(m_1, m_2, w) \equiv (m_1', m_2, w'),$$

where $(m_1, st_1) \leftarrow \text{DHInit}(pw)$, $w := \text{Comp}_i(pw, st_1, m_1, m_2)$, $(m_1', st_1') \leftarrow \text{SimInit}(\text{pp})$, $w' := \text{SimComp}_i(td, pw, st_1', m_1', m_2)$.

- ③ **Perfect Simulation for Responder:** For all $(\text{pp}, td) \leftarrow \text{DHSetup}$, all pw and all m_1 , it holds that

$$(m_1, m_2, w) \equiv (m_1, m_2', w'),$$

where $(m_2, st_2) \leftarrow \text{DHResp}(pw)$, $w := \text{Comp}_r(pw, st_2, m_1, m_2)$, $(m_2', st_2') \leftarrow \text{SimResp}(\text{pp})$, and $w' := \text{SimComp}_r(td, pw, st_2', m_1, m_2')$.

- ④ **Unique Password for Initiator:** The advantage function $\text{Adv}_{\text{Unipw}}^{\text{Init}}(\mathcal{A})$ is negligible for all PPT adversary $\mathcal{A}$ with oracle access to $\mathcal{O}_1(td^*, \dots)$, $\mathcal{O}_2(td^*, \dots)$ and $\mathcal{O}_1^*(td^*, st_1^*, m_1^*, \dots)$, where $\text{Adv}_{\text{Unipw}}^{\text{Init}}(\mathcal{A}) :=$

$$\Pr \left[\begin{array}{l} (\text{pp}^*, td^*) \leftarrow \text{DHSetup} \\ (m_1^*, st_1^*) \leftarrow \text{SimInit}(\text{pp}^*) \\ (pw, pw', m_2, w, w') \leftarrow \mathcal{A}^{\mathcal{O}_1, \mathcal{O}_2, \mathcal{O}_1^*}(\text{pp}^*, m_1^*) \end{array} : \begin{array}{l} pw \neq pw' \\ w = \text{SimComp}_i(td^*, pw, st_1^*, m_1^*, m_2) \\ w' = \text{SimComp}_i(td^*, pw', st_1^*, m_1^*, m_2) \end{array} \right],$$

$\mathcal{O}_1(td^*, pw, st_1, m_1, m_2, w) := [w \stackrel{?}{=} \text{SimComp}_i(td^*, pw, st_1, m_1, m_2)]$, $\mathcal{O}_2(td^*, pw, st_2, m_1, m_2, w) := [w \stackrel{?}{=} \text{SimComp}_r(td^*, pw, st_2, m_1, m_2)]$ and $\mathcal{O}_1^*(td^*, st_1^*, m_1^*, pw, m_2, w) := [w \stackrel{?}{=} \text{SimComp}_i(td^*, pw, st_1^*, m_1^*, m_2)]$.

- ⑤ **Unique Password for Responder:** The following advantage function is negligible for any PPT adversary $\mathcal{A}$ with oracle access to $\mathcal{O}_1(td^*, \dots)$, $\mathcal{O}_2(td^*, \dots)$, and $\mathcal{O}_2^*(td^*, st_2^*, m_2^*, \dots)$, where $\text{Adv}_{\text{Unipw}}^{\text{Resp}}(\mathcal{A}) :=$

$$\Pr \left[\begin{array}{l} (\text{pp}^*, td^*) \leftarrow \text{DHSetup} \\ (m_1, st_1) \leftarrow \mathcal{A}^{\mathcal{O}_1, \mathcal{O}_2, \mathcal{O}_2^*}(\text{pp}^*), \\ (m_2^*, st_2^*) \leftarrow \text{SimResp}(\text{pp}^*) \\ (pw, pw', w, w') \leftarrow \mathcal{A}^{\mathcal{O}_1, \mathcal{O}_2, \mathcal{O}_2^*}(st_1, \text{pp}^*, m_1) \end{array} : \begin{array}{l} pw \neq pw' \\ w = \text{SimComp}_r(td^*, pw, st_2^*, m_1, m_2^*) \\ w' = \text{SimComp}_r(td^*, pw', st_2^*, m_1, m_2^*) \end{array} \right],$$

$\mathcal{O}_1(td^*, pw, st_1, m_1, m_2, w) := [w \stackrel{?}{=} \text{SimComp}_i(td^*, pw, st_1, m_1, m_2)]$, $\mathcal{O}_2(td^*, pw, st_2, m_1, m_2, w) := [w \stackrel{?}{=} \text{SimComp}_r(td^*, pw, st_2, m_1, m_2)]$, and $\mathcal{O}_2^*(td^*, st_2^*, m_2^*, pw, m_1, w) := [w \stackrel{?}{=} \text{SimComp}_r(td^*, pw, st_2^*, m_1, m_2^*)]$.

Note that $A \equiv B$ in ② and ③ can be relaxed to statistical closeness between A and B .

3.2 UC Security of Full DH-Type PAKE

We will prove that any full DH-type PAKE has UC security in Theorem 1.

Theorem 1. *Any full DH-type PAKE per Def. 2, can securely emulate $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$ and hence achieve UC security in the random oracle model.*

According to Corollary 1 in [10], one can easily transform a PAKE protocol that securely emulates $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$ into a PAKE protocol that securely emulates $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$ by setting $pw' := \mathcal{P}[\mathcal{CP}|sid|ssid|pw]$, where pw is the password used in $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$ and pw' is used in $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$. By applying this result to Theorem 1, we have the following corollary.

Corollary 1. *Any full DH-type PAKE can securely emulate $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$ when the underlying password is replaced by $pw' := \mathcal{P}[\mathcal{CP}|sid|ssid|pw]$.*

Moreover, according to [3], by adding mutual explicit authentication to the PAKE protocol, its ideal functionality $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$ can be further strengthened to a slightly relaxed $\mathcal{F}_{\text{PAKE}}^3$, and any PAKE emulating relaxed $\mathcal{F}_{\text{PAKE}}$ implies perfect forward security in the IND model.

Proof of Theorem 1. Our goal is to construct Sim and show that $|\Pr[\mathbf{Real}_{\mathcal{Z},\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{Ideal}_{\mathcal{Z},\text{Sim}} \Rightarrow 1]|$ is negligible by employing a sequence of games, denoted as Game G_0 - G_8 . In this sequence, G_0 corresponds to $\mathbf{Real}_{\mathcal{Z},\mathcal{A}}$, while G_8 corresponds to $\mathbf{Ideal}_{\mathcal{Z},\text{Sim}}$. We aim to show that these adjacent games are indistinguishable from the view of $\mathcal{Z}$.

Game G_0 (Real World). This is the real experiment $\mathbf{Real}_{\mathcal{Z},\mathcal{A}}$. In this experiment, $\mathcal{Z}$ initializes a password for each party, sees the interactions among clients, servers and adversary $\mathcal{A}$, and also obtains the corresponding session keys of protocol instances. Here $\mathcal{A}$ may implement attacks like view, modify, insert, or drop messages over the network. We have $\Pr[\mathbf{Real}_{\mathcal{Z},\mathcal{A}} \Rightarrow 1] = \Pr[G_0 \Rightarrow 1]$.

Game G_1 (Simulations for Parties with pw). In this game, we introduce a simulator Sim who *additionally knows passwords* of all parties. Sim generates $\text{crs} := (\text{pp}, H)$ with $(\text{pp}, td) \leftarrow \text{DHSetup}$ and keeps td . Then it simulates the parties to generate transcripts for instances of the PAKE protocol, just like G_0 . With the knowledge of *passwords*, the simulations of the behaviors of all parties are perfect.

Moreover, Sim also simulates the random oracle H by maintaining a list $\mathcal{L}_H$. For a query x on $H(\cdot)$, if $(x, y) \in \mathcal{L}_H$, then Sim will return y as the reply. Otherwise, Sim will choose a random element y , record (x, y) in $\mathcal{L}_H$, and return y as the reply. By the ideal functionality of random oracles, Sim 's simulation for H is also perfect. So we have $\Pr[G_1 \Rightarrow 1] = \Pr[G_0 \Rightarrow 1]$.

Game G_2 (Simulations of Key without pw in case of Passive Attacks). G_2 is the same as G_1 , except for Sim 's simulation of generating Key in case of passive attacks.

³ In relaxed $\mathcal{F}_{\text{PAKE}}$, the adversary can try a late password test on an actively attacked session (at most once), but such sessions are guaranteed to terminate with an abort.

- When Sim simulates the session key $\text{Key} := H(pw, m_1, m_2, id, id', w)$, it checks the followings. If both m_1 and m_2 are generated by Sim via $(m_1, st_1) \leftarrow \text{DHInit}(pw)$ for instance $(\mathcal{P}, i)$ and $(m_2, st_2) \leftarrow \text{DHResp}(pw, m_1)$ for instance $(\mathcal{CP}, j)$ and $w = \text{Comp}_i(pw, st_1, m_1, m_2)$, then Sim samples $\text{Key} \leftarrow_s \mathcal{K}$ rather than $\text{Key} := H(pw, m_1, m_2, id, id', w)$. If $(\mathcal{P}, i)$ has identity id and $(\mathcal{CP}, j)$ has identity id' , then Sim sets the random Key as the session key for both $(\mathcal{P}, i)$ and $(\mathcal{CP}, j)$.

Accordingly, in G_2 , the generation of session keys does not need pw anymore in case of passive attacks.

G_2 is the same as G_1 , except there exists a pair of instance $(\mathcal{P}, i)$ and $(\mathcal{CP}, j)$ s.t. event $\text{Bad}_{i,j}$ happens.

Bad_{i,j}: $\mathcal{A}$ has issued hash query $(pw, m_1, m_2, id, id', w)$ on oracle H satisfying $w = \text{Comp}_i(pw, st_1, m_1, m_2)$, where $(m_1, st_1) \leftarrow \text{DHInit}(pw)$ is generated by Sim for $(\mathcal{P}, i)$, and $(m_2, st_2) \leftarrow \text{DHResp}(pw)$ is generated by Sim for $(\mathcal{CP}, j)$.

Define $\text{Bad} := \bigvee_{i,j} \text{Bad}_{i,j}$, So $|\Pr[G_2 \Rightarrow 1] - \Pr[G_1 \Rightarrow 1]| \leq \Pr[\text{Bad}] \leq \ell^2 \cdot \Pr[\text{Bad}_{i,j}]$, where ℓ is the maximal number of protocol instances. Next we show $\Pr[\text{Bad}_{i,j}] \leq \text{Adv}_{\text{key}}^{\text{OW}}(\mathcal{B}_{\text{ow}})$ with a security reduction: If $\text{Bad}_{i,j}$ happens, we construct algorithm $\mathcal{B}_{\text{ow}}$ breaking “① one-wayness of the DH-key” of the DH-type PAKE.

$\mathcal{B}_{\text{ow}}$ has public parameter pp , two messages m_1^*, m_2^* , and two oracles $\mathcal{O}_1(m_1^*, st_1^*, \dots)$, $\mathcal{O}_2(m_2^*, st_2^*, \dots)$. $\mathcal{B}_{\text{ow}}$ sets $\text{crs} := (\text{pp}, H)$, (m_1^*, id) and (m_2^*, id') as the output messages of $(\mathcal{P}, i)$ and $(\mathcal{CP}, j)$ respectively.

- If an initiator instance $(\mathcal{P}, i)$ with password pw receives a message $(m'_2, \hat{id})$, $\mathcal{B}_{\text{ow}}$ will search in the hash query list $\mathcal{L}_H$ to see whether there exists $(pw, m_1^*, m'_2, id, \hat{id}, [w, k]) \in \mathcal{L}_H$ s.t. $\mathcal{O}(m_1^*, st_1^*, pw, m'_2, w) = 1$. If yes, then $\mathcal{B}_{\text{ow}}$ sets $\text{Key} := k$. Otherwise, $\mathcal{B}_{\text{ow}}$ randomly samples $\text{Key} \leftarrow_s \mathcal{K}$ and will reprogram $H(pw, m_1^*, m'_2, id, \hat{id}, w) := \text{Key}$ if $\mathcal{A}$ later issues hash query $(pw, m_1^*, m'_2, id, \hat{id}, w)$ such that $\mathcal{O}(m_1^*, st_1^*, pw, m'_2, w) = 1$.
- If an responder instance $(\mathcal{CP}, j)$ receives a message (m'_1, id) , $\mathcal{B}_{\text{ow}}$ uses a similar strategy like initiator instance $(\mathcal{P}, i)$ to generate Key with help oracle $\mathcal{O}_2$ and pw . $\mathcal{B}_{\text{ow}}$ always keeps the initiator and the responder sharing the same session key in case of passive attacks.

$\mathcal{B}_{\text{ow}}$ can simulate round messages and session keys for other instances perfectly with help of passwords. So $\mathcal{B}_{\text{ow}}$ simulates G_2 perfectly for $\mathcal{Z}$. Finally, $\mathcal{B}_{\text{ow}}$ can search in $\mathcal{L}_H$ to find a hash query record $(pw, m_1^*, m_2^*, [id, id', w, k])$ such that $\mathcal{O}(m_1^*, st_1^*, pw, m_2^*, w) = 1$. If such a record exists, then $\text{Bad}_{i,j}$ happens since $\mathcal{O}(m_1^*, st_1^*, pw, m_2^*, w) = 1$ is equivalent to $w = \text{Comp}_i(pw, st_1^*, m_1^*, m_2^*)$. Then $\mathcal{B}_{\text{ow}}$ submits w as its own answer. Clearly $\mathcal{B}_{\text{ow}}$ wins if $\text{Bad}_{i,j}$ happens. So $\Pr[\text{Bad}_{i,j}] \leq \text{Adv}_{\text{key}}^{\text{OW}}(\mathcal{B}_{\text{ow}})$, and $|\Pr[G_2 \Rightarrow 1] - \Pr[G_1 \Rightarrow 1]| \leq \ell^2 \cdot \text{Adv}_{\text{key}}^{\text{OW}}(\mathcal{B}_{\text{ow}})$.

Game G_3 (Simulation of m_1 without pw). G_3 is the same as G_2 , except for Sim’s simulation of generating m_1 and w .

- Sim generates m_1 by $(m_1, st_1) \leftarrow \text{SimInit}(\text{pp})$ rather than $(m_1, st_1) \leftarrow \text{DHInit}(pw)$. Correspondingly, Sim computes $w := \text{SimComp}_i(td, pw, st_1, m_1, m_2)$ rather than $w := \text{Comp}_i(pw, st_1, m_1, m_2)$ in case of active attacks (recall that in case of passive attacks, Key is randomly chosen due to G'_2 , so pw or w are not needed at all).

By the “② Perfect Simulation for Initiator” property of the full DH-type PAKE protocol, G_3 is the same as G_2 . So we have $\Pr[G_3 \Rightarrow 1] = \Pr[G_2 \Rightarrow 1]$.

Game G_4 (simulation of m_2 without pw). G_4 is the same as G_3 , except for the simulation of generating m_2 and w .

- Sim generates m_2 by $(m_2, st_2) \leftarrow \text{SimResp}(\text{pp})$ rather than $(m_2, st_2) \leftarrow \text{DHResp}(pw)$. Correspondingly, Sim computes $w := \text{SimComp}_r(td_2, pw, st_2, m_1, m_2)$ rather than $w := \text{Comp}_r(pw, st_2, m_1, m_2)$ in case of active attacks (recall that in case of passive attacks, Key is randomly chosen due to G_2 , so pw or w are not needed at all).

By the “③ Perfect Simulation for Responder” property of the DH-type PAKE protocol, G_4 is the same as G_3 . So we have $\Pr[G_4 \Rightarrow 1] = \Pr[G_3 \Rightarrow 1]$.

Up to now, Sim does not use pw to generate round messages m_1, m_2 no matter in case of passive attacks or active attacks. Meanwhile, it does not use pw to generate DH-keys w in case of passive attacks, but still uses pw to compute DH-keys w in case of active attacks.

Game G_5 (Abort if There Exist Multiple Password Guesses for Initiator). G_5 is the same as G_4 , except Sim will abort if the following event Badl happens.

Badl: There exist an instance $(\mathcal{P}, i)$ and two hash queries $([pw], m_1, m_2, [id, id', w, k]) \in \mathcal{L}_H$ and $([pw'], m_1, m_2, [id, id', w', k']) \in \mathcal{L}_H$ s.t. $pw \neq pw'$, $w = \text{SimComp}_i(td, pw, st_1, m_1, m_2)$ and $w' = \text{SimComp}_i(td, pw', st_1, m_1, m_2)$, where m_1 is generated by Sim invoking $(m_1, st_1) \leftarrow \text{SimInit}(\text{pp})$ for $(\mathcal{P}, i)$.

We further define event Badl_j as the event that Badl happens for the j -th protocol instance. Then $\text{Badl} = \bigvee_{j=1}^{\ell} \text{Badl}_j$. If Badl does not happen, then G_5 is identical

to G_4 . So $|\Pr[G_5 \Rightarrow 1] - \Pr[G_4 \Rightarrow 1]| \leq \Pr[\text{Badl}] \leq \sum_{j=1}^{\ell} \Pr[\text{Badl}_j]$.

Now we show if Badl_j happens, then we can construct a reduction algorithm $\mathcal{B}_{init}$ breaking the “④ Unique Password for Initiator” property of the DH-type PAKE protocol.

$\mathcal{B}_{init}$ obtains pp^* and m_1^* from its own challenger, and has access three oracles $\mathcal{O}_1(td^*, \dots)$, $\mathcal{O}_2(td^*, \dots)$ and $\mathcal{O}_1^*(td^*, st_1^*, m_1^*, \dots)$, where pp^* is generated by DHSetup and m_1^* is generated by $(m_1^*, st_1^*) \leftarrow \text{SimInit}(\text{pp}^*)$. The goal of $\mathcal{B}_{init}$ is to find two valid password guesses for initiator, i.e., to find (pw, pw', m_2, w, w') s.t. $w = \text{SimComp}_i(td^*, pw, st_1^*, m_1^*, m_2)$ and $w' = \text{SimComp}_i(td^*, pw', st_1^*, m_1^*, m_2)$ but $pw \neq pw'$. To this end, $\mathcal{B}_{init}$ simulates G_5 to $\mathcal{Z}$ as follows.

$\mathcal{B}_{init}$ first sets $\text{crs} := (\text{pp}^*, H)$. For all but j protocol instances, $\mathcal{B}_{init}$ can simply generate m_1 by $(m_1, st_1) \leftarrow \text{SimInit}(\text{pp})$ for initiators and m_2 by $(m_2, st_2) \leftarrow \text{SimResp}(\text{pp})$ for responders, just like Sim does in G_5 . It can also simulate the generation of session key Key with the help of passwords and oracles $\mathcal{O}_1(td^*, \dots)$, $\mathcal{O}_2(td^*, \dots)$. Let us take a responder instance $(\mathcal{CP}, i)$ as an example. Let pw be the password used by $(\mathcal{CP}, i)$. Upon receiving a message (m_1, id) , $\mathcal{B}_{init}$ simulates Key for $(\mathcal{CP}, i)$ as follow. It first invokes $(m_2, st_2) \leftarrow \text{SimResp}(\text{pp})$ and outputs (m_2, id') . If there exists $(pw, m_1, m_2, [id, id', w, k]) \in \mathcal{L}_H$ s.t. $\mathcal{O}_2(td^*, pw, st_2, m_1, m_2, w) = 1$, then set $\text{Key} := k$. Otherwise, it randomly samples $\text{Key} \leftarrow_s \mathcal{K}$. If later $\mathcal{A}$ issues a hash query $(pw, m_1, m_2, id, id', w)$ such that $\mathcal{O}_2(td^*, pw, st_2, m_1, m_2, w) = 1$, then $\mathcal{B}_{init}$ reprograms $H(pw, m_1, m_2, id, id', w) := \text{Key}$ and stores $(pw, m_1, m_2, id, id', w, \text{Key})$ in $\mathcal{L}_H$. For initiator instance (not the j -th instance), $\mathcal{B}_{init}$ has a similar simulation of Key but with the help of $\mathcal{O}_1(td^*, \dots)$.

As for the j -th protocol instance, say $(\mathcal{P}^*, i^*)$, if $\mathcal{P}^*$ is not an initiator, then Badl_j does not happen and $\mathcal{B}_{init}$ aborts. Otherwise, $\mathcal{B}_{init}$ sets m_1^* and the identity id of $(\mathcal{P}^*, i^*)$ as the round message output of $(\mathcal{P}^*, i^*)$. Let pw^* be the password used by $(\mathcal{P}^*, i^*)$. Upon receiving (m_2, id') , $\mathcal{B}_{init}$ simulates the generation of session key as follows. If there exists $(pw^*, m_1^*, m_2, id, id', [w, k]) \in \mathcal{L}_H$ s.t. $\mathcal{O}_1^*(td^*, st_1^*, m_1^*, pw^*, m_2, w) = 1$, then set $\text{Key} := k$. Otherwise, it randomly samples $\text{Key} \leftarrow_s \mathcal{K}$. If later $\mathcal{A}$ issues hash query $(pw^*, m_1^*, m_2, id, id', w)$ such that $\mathcal{O}_1^*(td^*, st_1^*, m_1^*, pw^*, m_2, w) = 1$, then $\mathcal{B}_{init}$ reprograms $H(pw^*, m_1^*, m_2, id, id', w) := \text{Key}$ and stores $(pw^*, m_1^*, m_2, id, id', w, \text{Key})$ in $\mathcal{L}_H$.

Finally, $\mathcal{B}_{init}$ checks whether Badl_j happens: it goes through all records $([pw], m_1^*, [m_2, id, id', w]) \in \mathcal{L}_H$ and resorts to $\mathcal{O}_1^*(td^*, st_1^*, m_1^*, pw, m_2, w)$ to determine $[w \stackrel{?}{=} \text{SimComp}_r(td^*, pw, st_1^*, m_1^*, m_2)]$. If there are two records $(pw, m_1^*, m_2, id, id', w, k)$ and $(pw', m_1^*, m_2, id, id', w', k')$ such that $pw \neq pw'$ and $\mathcal{O}_1^*(td^*, st_1^*, m_1^*, pw, m_2, w) = \mathcal{O}_1^*(td^*, st_1^*, m_1^*, pw', m_2, w') = 1$, then Badl_j happens. In this case, $\mathcal{B}_{init}$ submits (pw, pw', m_2, w, w') as its answer to its challenger.

Note that $\mathcal{B}_{init}$ perfectly simulates G_4 for $\mathcal{Z}$, just like Sim . If Badl_j happens, then $\mathcal{B}_{init}$ breaks the “④ Unique Password for Initiator” property. So we have $\Pr[\text{Badl}_j] \leq \text{Adv}_{\text{Unipw}}^{\text{Init}}(\mathcal{B}_{init})$ and

$$|\Pr[\text{G}_5 \Rightarrow 1] - \Pr[\text{G}_4 \Rightarrow 1]| \leq \Pr[\text{Badl}] \leq \sum_{j=1}^{\ell} \Pr[\text{Badl}_j] \leq \ell \cdot \text{Adv}_{\text{Unipw}}^{\text{Init}}(\mathcal{B}_{init}).$$

Game G_6 (Abort if There Exist Multiple Password Guesses for Responder). G_6 is the same as G_5 , except Sim will abort if the following event BadR happens.

BadR: There exist an instance $(\mathcal{P}, i)$ and two hash queries $([pw], m_1, m_2, [id, id', w, k]) \in \mathcal{L}_H$ and $([pw'], m_1, m_2, [id, id', w', k']) \in \mathcal{L}_H$ s.t. $pw \neq pw'$, $w = \text{SimComp}_r(td, pw, st_2, m_1, m_2)$ and $w' = \text{SimComp}_r(td, pw', st_2, m_1, m_2)$, where (m_2, id') is generated by Sim invoking $(m_2, st_2) \leftarrow \text{SimResp}(\text{pp})$ for $(\mathcal{P}, i)$.

We further define event BadR_j as the event that BadR happens for the j -th protocol instances. With a similar argument as G_5 , we have

$$|\Pr[G_6 \Rightarrow 1] - \Pr[G_5 \Rightarrow 1]| \leq \Pr[\text{BadR}] \leq \sum_{j=1}^{\ell} \Pr[\text{BadR}_j] \leq \ell \cdot \text{Adv}_{\text{Unipw}}^{\text{Resp}}(\mathcal{B}_{\text{resp}}).$$

Game G_7 (Simulations of Key without pw in case of Active Attacks). G_7 is the same as G_6 , except for the simulation of Key in case of active attacks.

- For an initiator instance $(\mathcal{P}, i)$, suppose that (m_1, id) is the message generated by Sim for $(\mathcal{P}, i)$, and (m_2, id') is a message received from $\mathcal{A}$. We assume that (m_2, id') is not generated by Sim, and hence (m_2, id') results from $\mathcal{A}$'s active attack. In this case, Sim searches in the hash list $\mathcal{L}_H$. If there exists $([pw], m_1, m_2, id, id', [w, k]) \in \mathcal{L}_H$ s.t. $w = \text{SimComp}_i(td, pw, st_1, m_1, m_2)$ (at most one due to G_6), then Sim checks whether pw is the password used by $(\mathcal{P}, i)$. if yes, then $\mathcal{A}$ must have implemented a successful active attack on $(\mathcal{P}, i)$, so Sim sets $\text{Key} := k$. If there exists no such record (i.e., $\mathcal{A}$'s active attack fails), Sim randomly samples $\text{Key} \leftarrow_s \mathcal{K}$ and reprograms $H(pw, m_1, m_2, id, id', w) := \text{Key}$ for $\mathcal{A}$'s later hash query $(pw, m_1, m_2, id, id', w)$ s.t. $w = \text{SimComp}_i(td, pw, st_1, m_1, m_2)$.
- For a responder instance $(\mathcal{CP}, j)$, suppose that (m_2, id') is the message generated by Sim for $(\mathcal{CP}, j)$, and (m_1, id) is the message received from $\mathcal{A}$. We assume that (m_1, id) is not generated by Sim, and hence (m_1, id) results from $\mathcal{A}$'s active attack. In this case, Sim searches in the hash list $\mathcal{L}_H$. If there exists $([pw], m_1, m_2, id, id', [w, k]) \in \mathcal{L}_H$ s.t. $w = \text{SimComp}_r(td, pw, st_2, m_1, m_2)$ (at most one due to G_6), then Sim checks whether pw is the password used by $(\mathcal{CP}, j)$. If yes, Sim sets $\text{Key} := k$. If there exists no such record, Sim randomly samples $\text{Key} \leftarrow_s \mathcal{K}$ and reprograms $H(pw, m_1, m_2, id, id', w) := \text{Key}$ for $\mathcal{A}$'s later hash query $(pw, m_1, m_2, id, id', w)$ s.t. $w = \text{SimComp}_r(td, pw, st_2, m_1, m_2)$.

According to the ideal functionality of random oracle and the reprogramming technique, we know the changes are conceptual. So $\Pr[G_7 \Rightarrow 1] = \Pr[G_6 \Rightarrow 1]$.

Note that Sim almost gets rid of using pw when dealing with $\mathcal{A}$'s active attacks, except that Sim still uses the party's password pw to check the correctness of pw' in record $(pw', \dots)$ of $\mathcal{L}_H$ so as to identify $\mathcal{A}$'s successful active attacks during the session key generation.

Game G_8 (Integration of Sim with $\mathcal{F}_{\text{bPAKELE}}$). In the final game G_8 , Sim completely gets rid of pw . More precisely, Sim accesses $\mathcal{F}_{\text{bPAKELE}}$ via interfaces `PassiveNewKey`, `ActiveNewKey` and `LateTestPwd` to check the correctness of passwords and assigns session keys for party instances. The full description of Sim is shown in our full version [29].

We consider the following two cases, covering both passive and active attacks.

Passive Attacks: In this case, Sim generates session keys via the `PassiveNewKey` interface from $\mathcal{F}_{\text{bPAKELE}}$.

- For an instance $(\mathcal{P}, i)$ that receives a message (m_1, id) or (m_2, id') where (m_1, id) or (m_2, id') is generated by Sim for some instance $(\mathcal{CP}, j)$, Sim sets $ssid := m_1 | m_2 | id | id'$ and sends $(\text{PassiveNewKey}, sid, \mathcal{P}, i, \mathcal{CP}, j, ssid)$ to $\mathcal{F}_{\text{bPAKELE}}$.

According to its ideal functionality, $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$ assigns a uniform session key to instance $(\mathcal{P}, i)$ and keeps the session keys of $(\mathcal{P}, i)$ and $(\mathcal{CP}, j)$ consistent. Therefore, G_8 and G_7 are the same in this case.

Active Attacks: In this case, Sim generates session keys via **ActiveNewKey** interface and may reprogram the random oracle H with help of the **LateTestPwd** interface.

- For an initiator instance $(\mathcal{P}, i)$, suppose that (m_1, id) is generated by Sim invoking $(m_1, st_1) \leftarrow \text{SimInit}(\text{pp})$ for $(\mathcal{P}, i)$. Upon receiving message (m_2, id') that is generated by $\mathcal{A}$, Sim resorts to $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$ to generate session key for $(\mathcal{P}, i)$. Sim searches in $\mathcal{L}_H$ to find $([pw], m_1, m_2, id, id', [w, K]) \in \mathcal{L}_H$ s.t. $w = \text{SimComp}_i(td, pw, st_1, m_1, m_2)$. If there exists such a record (at most one), then Sim sends $(\text{ActiveNewKey}, sid, \mathcal{P}, i, pw, K, ssid, id')$ to $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$. If there exists no such record, then Sim sends $(\text{ActiveNewKey}, sid, \mathcal{P}, i, \perp, \perp, ssid, id')$. Later, if $\mathcal{A}$ issues a hash query on $([pw], m_1, m_2, id, id', [w])$ for H s.t. $w = \text{SimComp}_i(td, pw, st_1, m_1, m_2)$, then Sim sends $(\text{LateTestPwd}, sid, \mathcal{P}, i, ssid, pw)$ to $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$ and obtains Key' . Then Sim programs $H(pw, m_1, m_2, id, id', w) := \text{Key}'$.
- For a responder instance $(\mathcal{CP}, j)$, suppose that (m_2, id') is generated by Sim invoking $(m_2, st_2) \leftarrow \text{SimResp}(\text{pp})$ for $(\mathcal{CP}, j)$. Upon receiving message (m_1, id) that is generated by $\mathcal{A}$, Sim uses **ActiveNewKey** and **LateTestPwd** to generate session key Key in a similar way as above.

According to the specification of **ActiveNewKey**, if pw in the record is the correct password, then $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$ sets $\text{Key} := H(pw, m_1, m_2, id, id', w)$. Otherwise $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$ randomly samples $\text{Key} \leftarrow_s \mathcal{K}$. Furthermore, Sim can reprogram $H(pw, m_1, m_2, id, id', w)$ with help of **LateTestPwd** to keep consistence. In this case, G_8 and G_7 are the same.

So we have $\Pr[G_8 \Rightarrow 1] = \Pr[G_7 \Rightarrow 1]$.

Here we assume the $ssid = m_1|m_2|id|id'$ in **PassiveNewKey/ActiveNewKey** is unique for $sid, (\mathcal{P}, i)$ and $(\mathcal{CP}, j)$. This is reasonable since those m_1 outputted by **SimInit** are hardly collide, and so is m_2 by **SimResp**.

Now that Sim completely gets rid of pw in the simulation, G_8 is exactly $\text{Ideal}_{\mathcal{Z}, \text{Sim}}$. Therefore, $\Pr[\text{Ideal}_{\mathcal{Z}, \text{Sim}} \Rightarrow 1] = \Pr[G_8]$.

Finally, by combining all the statements across G_0 - G_8 , we know that

$$\begin{aligned} & |\Pr[\text{Real}_{\mathcal{Z}, \mathcal{A}} \Rightarrow 1] - \Pr[\text{Ideal}_{\mathcal{Z}, \text{Sim}} \Rightarrow 1]| \\ & \leq \ell \cdot (\ell \cdot \text{Adv}_{\text{key}}^{\text{ow}}(\mathcal{B}_{\text{ow}}) + \text{Adv}_{\text{Unipw}}^{\text{Init}}(\mathcal{B}_{\text{init}}) + \text{Adv}_{\text{Unipw}}^{\text{Resp}}(\mathcal{B}_{\text{resp}})). \end{aligned}$$

□

3.3 Hybrid DH-Type PAKE via Parallel Composition

Suppose $\text{PAKE}_1, \dots, \text{PAKE}_n$ are DH-type PAKE schemes with $\text{PAKE}_j = (\text{DHSetup}_j, \text{DHInit}_j, \text{DHResp}_j, \text{Comp}_{ij}, \text{Comp}_{rj})$ for $j \in [n]$. Then we can compose $\text{PAKE}_1, \dots, \text{PAKE}_n$ in parallel to obtain a new Hybrid DH-type PAKE scheme HPAKE_{DH} . The full description of scheme HPAKE_{DH} is given in Fig. 7.

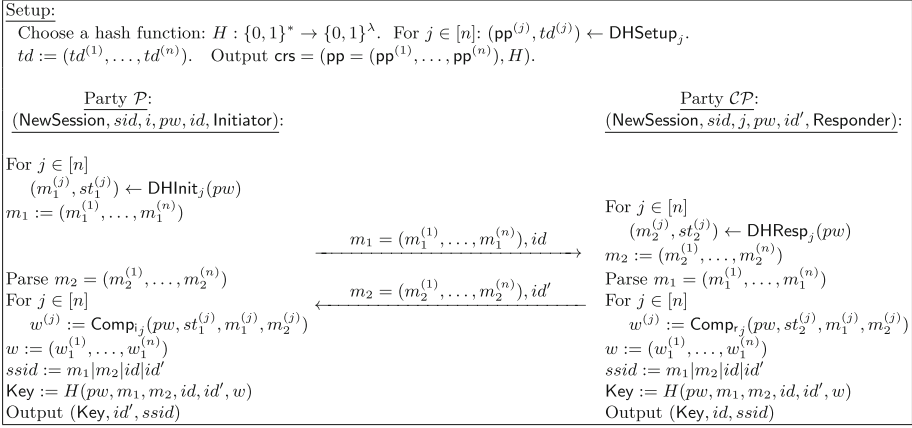


Fig. 7. Hybrid DH-type PAKE HPAKE_{DH} constructed via parallel composition of $\text{PAKE}_1, \dots, \text{PAKE}_n$, where $\text{PAKE}_j = (\text{DHSetup}_j, \text{DHInit}_j, \text{DHResp}_j, \text{Comp}_{i_j}, \text{Comp}_{r_j})$ for $j \in [n]$ are DH-type PAKEs.

Theorem 2. *The hybrid PAKE scheme HPAKE_{DH} in Fig. 7, constructed via parallel composition of $\text{PAKE}_1, \dots, \text{PAKE}_n$, is a full DH-type PAKE, as long as one component PAKE_j is a full DH-type PAKE and the others $\{\text{PAKE}_i\}_{i \in [n] \setminus \{j\}}$ are DH-type PAKE (not necessarily full) with statistical properties of ②, ③.*

The proof sketch has shown in technique overview in Sect. 1. So we move the full proof in our full version [29].

4 Hybrid PAKE via Serial Composition of DH-Type PAKE and Other PAKE

Given a DH-type PAKE scheme $\text{PAKE}_1 = (\text{DHSetup}, \text{DHInit}, \text{DHResp}, \text{Comp}_i, \text{Comp}_r)$ and a (two-round) PAKE scheme $\text{PAKE}_2 = (\text{Setup}, \text{Init}, \text{Resp}, \text{Deri})$, we show how to use them to construct a Hybrid PAKE scheme HPAKE via serial composition of PAKE_1 and PAKE_2 . In the serial composition, PAKE_1 will obtain its session key $k_1 | k_2$, and then the first part k_1 is used as the password of PAKE_2 . Next, PAKE_2 will obtain its session key k_3 . Finally, the session key of HPAKE is computed by the hash value of the second part k_2 of PAKE_1 's session key, PAKE_2 's the session key, and the transcript of HPAKE , as shown in Fig. 8.

Let $\mathcal{F}_{\text{PAKE}_1}^{\text{leaky}}$ be the (lazy extraction version of) ideal functionality of PAKE_1 providing interfaces **NewSession1**, **Testpw1**, **RegisterTest1**, **LateTestPwd1** and **NewKey1**. Let $\mathcal{F}_{\text{PAKE}_2}$ be the ideal functionality of PAKE_2 providing interfaces **NewSession2**, **Testpw2**, **RegisterTest2**, **LateTestPwd2** and **NewKey2**.

Let $\mathcal{F}_{\text{HPAKE}}^{\text{leaky}}$ denote the leaky and lazy extraction version of ideal functionality of HPAKE providing interfaces **NewSession**, **Testpw**, **RegisterTest**, **LateTestPwd**, and **NewKey**, and $\mathcal{F}_{\text{HPAKE}}$ denote the normal ideal functionality of HPAKE , providing interfaces **NewSession**, **Testpw**, and **NewKey**.

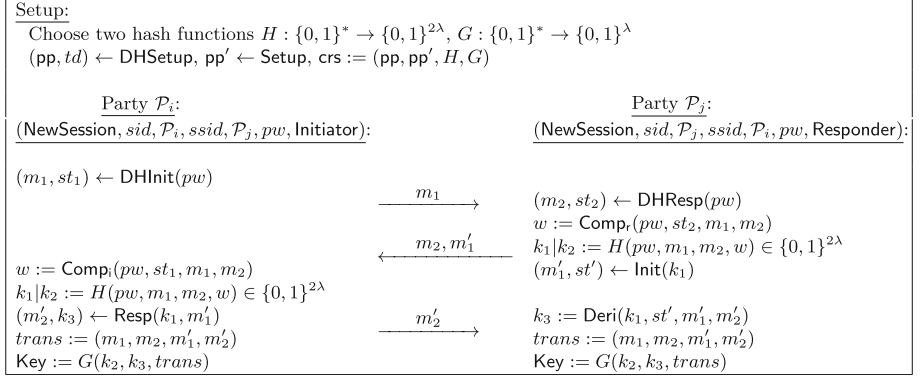


Fig. 8. Our Hybrid PAKE scheme HPAKE constructed via serial composition of a DH-type PAKE₁ and another PAKE₂. Here we use a two-round PAKE₂ for simplicity, but any (multi-round) PAKE₂ works as well.

In the following theorems, we prove the UC security of HPAKE can rely on either the UC security of PAKE₁ or the UC security of PAKE₂.

Theorem 3. *The PAKE scheme HPAKE in Fig. 8 securely emulates $\mathcal{F}_{\text{PAKE}^{\text{leaky}}}$ if the underlying PAKE₁ = (Setup, DHInit, DHResp, Comp_i, Comp_r) securely emulates $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$, and hash function G works as random oracles.*

Theorem 4. *The PAKE scheme HPAKE in Fig. 8 securely emulates $\mathcal{F}_{\text{PAKE}}$ if the underlying PAKE₂ = (Setup, Init, Resp, Deri) securely emulates $\mathcal{F}_{\text{PAKE}}$, PAKE₁ = (Setup, DHInit, DHResp, Comp_i, Comp_r) is a DH-type PAKE satisfying statistical properties ②③ as per Def. 2, and hash functions H, G are (quantum-accessible) random oracles⁴.*

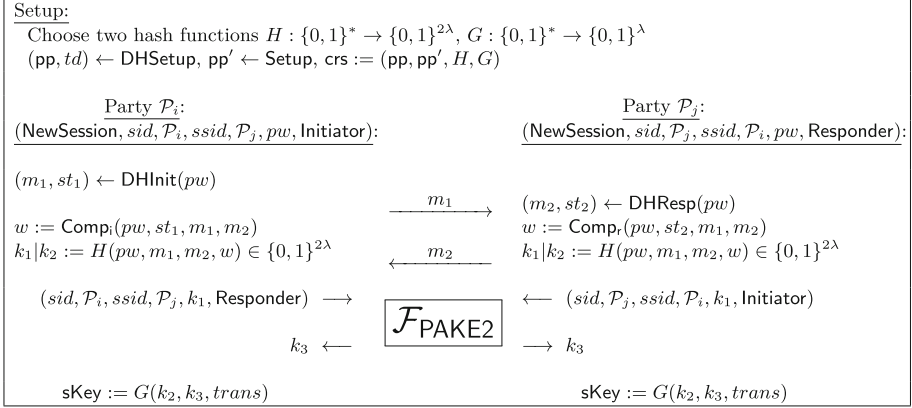
Proof of Theorem 3. The proof sketch has shown in technique overview in Sect. 1. So we move the full proof in in our full version [29].

Proof of Theorem 4. We will prove it in the $\mathcal{F}_{\text{PAKE}_2}$ -hybrid model, where the underlying PAKE₂ protocol is replaced by $\mathcal{F}_{\text{PAKE}_2}$, as shown in Fig. 9. Our goal is to construct a simulator Sim and employ a series of games to show that $\left| \Pr \left[\text{Real}_{\mathcal{Z}, \mathcal{A}}^{\mathcal{F}_{\text{PAKE}_2}} \Rightarrow 1 \right] - \Pr \left[\text{Ideal}_{\mathcal{Z}, \text{Sim}[\mathcal{F}_{\text{PAKE}_2}]} \Rightarrow 1 \right] \right|$ is negligible.

Game G_0 (Real World). This is the $\text{Real}_{\mathcal{Z}, \mathcal{A}}^{\mathcal{F}_{\text{PAKE}_2}}$ world where adversary $\mathcal{A}$ can access ideal functionality $\mathcal{F}_{\text{PAKE}_2}$ via interfaces like Testpw2 and NewKey2.

Game G_1 (Simulating each Party with Password pw). In this game, we introduce a simulator Sim who *additionally knows passwords* of all parties. Sim generates $\text{crs} := (pp, pp', H, G)$ with $(pp, td) \leftarrow \text{DHSetup}$ and $pp' \leftarrow \text{Setup}$ and keeps td . Then with help of passwords, it simulates the parties to generate transcripts (m_1, m_2) and session keys sKey for instances of the HPAKE protocol, just

⁴ It means that hash functions in (hybrid) PAKE are modeled as quantum random oracles, i.e., the adversary has quantum access to hash functions.

Fig. 9. HPAKE in $\mathcal{F}_{\text{PAKE2}}$ -hybrid model.

like G_0 . Sim also simulates the underlying $\mathcal{F}_{\text{PAKE2}}$ by itself with passwords. With the knowledge of *passwords*, Sim's simulations of $\mathcal{F}_{\text{PAKE2}}$ and the behaviors of all parties are perfect. We have $\Pr[G_1 \Rightarrow 1] = \Pr[G_0 \Rightarrow 1]$.

Game G_2 (Simulating transcripts m_1, m_2 without pw). G_2 is the same as G_1 , except for Sim's simulation of m_1 and m_2 .

- For an initiator instance $(\mathcal{P}_i, ssid)$, Sim generates m_1 by $(m_1, st_1) \leftarrow \text{SimInit}(pp)$ rather than $(m_1, st_1) \leftarrow \text{DHInit}(pw)$. Correspondingly, when receiving the second message m_2 , Sim computes $w := \text{SimComp}_i(td, pw, st_1, m_1, m_2)$ rather than $w := \text{Comp}_i(pw, st_1, m_1, m_2)$.
- For a responder instance $(\mathcal{P}_j, ssid)$, upon receiving a first-round message m_1 , Sim generates m_2 by $(m_2, st_2) \leftarrow \text{SimResp}(pp)$ rather than $(m_2, st_2) \leftarrow \text{DHResp}(pw)$. Correspondingly, Sim computes $w := \text{SimComp}_r(td, pw, st_2, m_1, m_2)$ rather than $w := \text{Comp}_r(pw, st_2, m_1, m_2)$.

By the “② Perfect Simulation for Initiator” and “③ Perfect Simulation for Responder” properties of the DH-type PAKE_1 , G_2 is identical to G_1 . So we have

$$\Pr[G_2 \Rightarrow 1] = \Pr[G_1 \Rightarrow 1].$$

Game G_3 (Resorting to Online-Extractable QRO Simulator $\mathcal{S}$ for H). In G_3 , to simulate random oracle H , Sim invokes the random oracle simulator $\mathcal{S} = (\mathcal{S}.RO, \mathcal{S}.E)$ as per online-extractable simulator [20] (See Theorem 5 in our full version [29]) with respect to function $f : f(x, y_1|y_2) := y_1$, and it also extracts the hash query from the value k_1 . More precisely, Sim invokes $\mathcal{S}.RO(x)$ to simulate the quantum random oracle H . For each of $\mathcal{A}$'s queries $(\text{Testpw2}, sid, \mathcal{P}, ssid, k_1)$ to the simulated $\mathcal{F}_{\text{PAKE2}}$, Sim invokes $\mathcal{S}.E(k_1)$ to extract $(\hat{p}w, \hat{m}_1, \hat{m}_2, \hat{w})$.

The only difference between G_3 and G_2 is the simulation of oracle H . We define the function $f(x, y_1|y_2) := y_1$ where $y_1|y_2 \in \{0, 1\}^{2\lambda}$. It is easy to see $\Gamma(f) = 2^\lambda$ and $\Gamma'(f) = 2^\lambda$ (See [20] for the definition of Γ and Γ' , see also our full version [29]). The invocation of $\mathcal{S}.E(k_1)$ corresponds to Sim conducting measurements via $\mathcal{S}.E(k_1)$ w.r.t. f .

Since Sim invokes $\mathcal{S}.E$ at most ℓ times (limited by the number of instances), and $\mathcal{A}$ issues at most q quantum random oracle queries, the online-extractable technique shows that $|\Pr[G_3 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| \leq \frac{8\ell(q+\ell)}{\sqrt{2}^{2\lambda-1}}$.

Game G_4 (Answer Testpw2 with Help of Online-Extractable QRO Simulator $\mathcal{S}$). In G_4 , Testpw2 is answered by Sim with help of $\mathcal{S} = (\mathcal{S}.RO, \mathcal{S}.E)$.

- Upon receiving the $(\text{Testpw2}, \text{sid}, \mathcal{P}, \text{ssid}, k'_1)$ query from $\mathcal{A}$ to the underlying $\mathcal{F}_{\text{PAKE2}}$, Sim invokes $\mathcal{S}.E(k'_1)$ to extract $(\hat{p}w, \hat{m}_1, \hat{m}_2, \hat{w})$. If $\hat{p}w = pw$, $(\hat{m}_1, \hat{m}_2)$ are transcripts sent and received by $(\mathcal{P}, \text{ssid})$, and $\hat{w} = \text{SimComp}_i(td, \hat{p}w, st_1, \hat{m}_1, \hat{m}_2)$ or $\hat{w} = \text{SimComp}_r(td, \hat{p}w, st_2, \hat{m}_1, \hat{m}_2)$, then Sim returns “correct guess” to $\mathcal{A}$. Otherwise, Sim returns “wrong guess” to $\mathcal{A}$.

Recall that in G_3 , for a $(\text{Testpw2}, \text{sid}, \mathcal{P}, \text{ssid}, k'_1)$ query, Sim computes $w := \text{SimInit}(td, pw, st_1, m_1, m_2)$ or $w := \text{SimResp}(td, pw, st_2, m_1, m_2)$, and $k_1|k_2 := H(pw, m_1, m_2, w)$, where pw is the correct password of $(\mathcal{P}, \text{ssid})$ and m_1, m_2 are the messages sent and received by $(\mathcal{P}, \text{ssid})$. If $k'_1 = k_1$, then Sim returns “correct guess”. Otherwise, Sim returns “wrong guess”.

If $(\hat{p}w, \hat{m}_1, \hat{m}_2, \hat{w}) = (pw, m_1, m_2, w)$, G_4 is the same as G_3 . Now we consider the case of $(\hat{p}w, \hat{m}_1, \hat{m}_2, \hat{w}) \neq (pw, m_1, m_2, w)$. Given $k_1|k_2 := H(pw, m_1, m_2, w)$, we further consider the following two cases.

Case I: $(\hat{p}w, \hat{m}_1, \hat{m}_2, \hat{w}) \neq (pw, m_1, m_2, w)$ and $k'_1 = k_1$. This case is bounded by the online-extraction theorem [20] (c.f. Thm. 5(b) in our full version [29]).

Case II: $(\hat{p}w, \hat{m}_1, \hat{m}_2, \hat{w}) \neq (pw, m_1, m_2, w)$ and $k'_1 \neq k_1$. In this case, Sim in G_3 must return “wrong guess”. If Sim in G_4 returns “correct guess”, it must hold $\hat{p}w = pw$ and $(\hat{m}_1, \hat{m}_2) = (m_1, m_2)$. Recall that SimComp_i and SimComp_r are deterministic functions, so $\hat{w} = w$, which is contradictory to $(\hat{p}w, \hat{m}_1, \hat{m}_2, \hat{w}) \neq (pw, m_1, m_2, w)$. Therefore, G_4 is the same as G_3 in this case.

According to the online-extraction theorem [20] (c.f. Theorem 5(b) in our full version [29]), we have $|\Pr[G_4 \Rightarrow 1] - \Pr[G_3 \Rightarrow 1]| \leq \frac{8\ell(q+1)}{\sqrt{2}^{2\lambda-1}} + \frac{40e^2(q+\ell+1)^3+2}{2^\lambda}$.

Game G_5 (Simulating Session Keys sKey). G_5 is the same as G_4 , except for the simulation of session keys sKey.

- Sim checks whether $\mathcal{A}$ ever issued a $(\text{Testpw2}, \text{sid}, \mathcal{P}, \text{ssid}, k'_1)$ query to the underlying $\mathcal{F}_{\text{PAKE2}}$ and obtained “correct guess”.
 - (1) If yes, then Sim computes $\text{sKey} := G(k_2, k_3, \text{trans})$ just like G_4 .
 - (2) If no and there exists a party $\mathcal{CP}$ with session key sKey' that shares the same m_1, m_2, k_3 , then Sim sets $\text{sKey} := \text{sKey}'$. Due to the correctness of HPAKE, G_5 and G_4 are the same in this case.
 - (3) If no and no other instance shares m_1, m_2, k_3 with $(\mathcal{P}, \text{ssid})$, Sim randomly samples $\text{sKey} \leftarrow \mathcal{K}$.

Note that (1) means a successful active attack, (2) means a passive attack, and (3) means a passive attack or an unsuccessful active attack.

Recall that in case (3), we have $\text{sKey} := G(k_2, k_3, \text{trans})$ in G_4 . So G_5 and G_4 are the same except case (3) happens. According to the specification of $\mathcal{F}_{\text{PAKE2}}$, $\mathcal{F}_{\text{PAKE2}}$ will output a random $k_3 \leftarrow_{\$} \{0, 1\}^\lambda$ in case (3). Then the O2H Lemma [15] (See also Lemma 1 in our full version [29]) guarantees that the function $G(\cdot, k_3, \cdot)$ is a pseudo-random function in QROM, except with probability $\frac{2q}{\sqrt{2}^\lambda}$. Then different transcripts trans in different instances make sure that the outputs of quantum random oracle $G(k_2, k_3, \text{trans})$ are independent and uniformly distributed in case (3). So $|\Pr[G_5 \Rightarrow 1] - \Pr[G_4 \Rightarrow 1]| \leq \frac{2q}{\sqrt{2}^\lambda}$.

Game G_6 (Integration of Sim with $\mathcal{F}_{\text{PAKE}}$). Up to now only Sim's simulation of transcripts (m_1, m_2) does not use password. In G_6 , Sim will completely get rid of passwords. It will use interfaces of $\mathcal{F}_{\text{PAKE}}$ (in the ideal world for HPAKE) to answer $\mathcal{A}$'s **Testpw2** and **NewKey2** queries to $\mathcal{F}_{\text{PAKE2}}$ without using passwords. As long as no password is needed for answering **Testpw2** query to $\mathcal{F}_{\text{PAKE2}}$, Sim does not need passwords for generating session key sKey either (see G_5).

- For (**NewSession**, $\text{sid}, \mathcal{P}, \text{ssid}, \mathcal{CP}, \cdot, \text{role}$) from $\mathcal{P}$, if this is the first **NewSession** query on ssid , or this is the second **NewSession** query on ssid and $(\mathcal{CP}, \text{ssid}, \mathcal{P}, \cdot)$ exists, then Sim records $(\mathcal{P}, \text{ssid}, \mathcal{CP}, \cdot)$ and marks it **fresh**.
- For $\mathcal{A}$'s query (**Testpw2**, $\text{sid}, \mathcal{P}, \text{ssid}, k'_1$) to $\mathcal{F}_{\text{PAKE2}}$, if there is a **fresh** record $(\mathcal{P}, \text{ssid}, [\mathcal{CP}, \cdot])$, Sim first invokes $\mathcal{S}.E(k'_1)$ to extract $(\hat{p}w, \hat{m}_1, \hat{m}_2, \hat{w})$. If $(\hat{m}_1, \hat{m}_2)$ are not transcripts sent and received by $(\mathcal{P}, \text{ssid})$, or $\hat{w} \neq \text{SimComp}_p(td, \hat{p}w, st_1, \hat{m}_1, \hat{m}_2)$ and $\hat{w} \neq \text{SimComp}_r(td, \hat{p}w, st_2, \hat{m}_1, \hat{m}_2)$, then Sim marks the record **interrupted** and replies with “wrong guess” to $\mathcal{A}$. Otherwise, Sim issues query (**Testpw**, $\text{sid}, \mathcal{P}, \text{ssid}, \hat{p}w$) to $\mathcal{F}_{\text{PAKE}}$. If $\mathcal{F}_{\text{PAKE}}$ returns “correct guess”, then Sim marks the record as **compromised** and replies with “correct guess” to $\mathcal{A}$. Otherwise, Sim marks the record as **interrupted** and replies with “wrong guess” to $\mathcal{A}$.
- For $\mathcal{A}$'s query (**NewKey2**, $\text{sid}, \mathcal{P}, \text{ssid}, \text{Key}^*$) to $\mathcal{F}_{\text{PAKE2}}$, if record $(\mathcal{P}, \text{ssid}, [\mathcal{CP}, \cdot])$ is not marked **completed**, do the followings.
 - If the record is **compromised**, Sim sets $k_3 := \text{Key}^*$, computes $\text{sKey}^* = G(k_2, k_3, \text{trans})$.
 - In all other cases, Sim sets $\text{sKey}^* = \perp$.
 Sim issues (**NewKey**, $\text{sid}, \mathcal{P}, \text{ssid}, \text{sKey}^*$) to $\mathcal{F}_{\text{PAKE}}$ and marks the record $(\mathcal{P}, \text{ssid}, \mathcal{CP}, \cdot)$ **completed**.

According to the specification of $\mathcal{F}_{\text{PAKE}}$, for (**Testpw**, $\text{sid}, \mathcal{P}, \text{ssid}, \hat{p}w$) query, $\mathcal{F}_{\text{PAKE}}$ returns “correct guess” iff $\hat{p}w = pw$. Therefore, Sim's simulation of answer to **Testpw2** in G_6 is the same as that in G_5 , but without pw . At the same time, if $\mathcal{A}$'s **Testpw2** query results in “correct guess”, then the instance is compromised. In G_6 , we have $\text{sKey}^* = G(k_2, k_3, \text{trans})$ for compromised instance. Accordingly, for (**NewKey**, $\text{sid}, \mathcal{P}, \text{ssid}, \text{sKey}^*$) query, $\mathcal{F}_{\text{PAKE}}$ will set the session key of $(\mathcal{P}, \text{ssid})$ with $\text{sKey} := \text{sKey}^* = G(k_2, k_3, \text{trans})$. For uncorrupted $(\mathcal{P}, \text{ssid})$, $\mathcal{F}_{\text{PAKE}}$ will either keep sKey consistent between partnered instances in case of passive attacks

or sample a random key as the session key $\mathbf{sKey}$ in other cases. So the simulation of session keys $\mathbf{sKey}$ in G_6 is also identical to G_5 . So $\Pr[G_6 \Rightarrow 1] = \Pr[G_5 \Rightarrow 1]$.

Now $\mathbf{Sim}$ can simulate the whole experiment only with access to the ideal functionality $\mathcal{F}_{\text{PAKE}}$ and does not need passwords anymore. The full description of simulator $\mathbf{Sim}$ is shown in our full version [29]. G_6 is exactly $\mathbf{Ideal}_{\mathcal{Z}, \mathbf{Sim}[\mathcal{F}_{\text{PAKE}}]}$. Therefore, $\Pr[\mathbf{Ideal}_{\mathcal{Z}, \mathbf{Sim}[\mathcal{F}_{\text{PAKE}}]} \Rightarrow 1] = \Pr[G_6 \Rightarrow 1]$.

Finally, by combining all the statements across G_0 - G_6 , we have

$$\left| \Pr[\mathbf{Real}_{\mathcal{Z}, \mathcal{A}}^{\mathcal{F}_{\text{PAKE2}}}] - \Pr[\mathbf{Ideal}_{\mathcal{Z}, \mathbf{Sim}[\mathcal{F}_{\text{PAKE2}}]}] \right| \leq \frac{16\ell(q + \ell)}{\sqrt{2^{\lambda} - 1}} + \frac{40e^2(q + \ell + 1)^3 + 2}{2^{\lambda}} + \frac{2q}{\sqrt{2^{\lambda}}}.$$

5 Instantiations

We show that PAKE schemes SPAKE2 [7], CrsX-GA-PAKE [4, 24], and TBPEKE [31] are all fall into the framework of full DH-type PAKE. Due to lack of space, we put the formal proofs in our full version [29]. Recall that Abdalla et al. proved that SPAKE2 and TBPEKE can securely emulate $\mathcal{F}_{\text{PAKE}^{\text{LE}}}$ in [3]. In comparison, our results are stronger since we prove that SPAKE2, TBPEKE, and CrsX-GA-PAKE can securely emulate a better ideal functionality $\mathcal{F}_{\text{bPAKE}^{\text{LE}}}$. Moreover, we give the first security proof for CrsX-GA-PAKE in the UC framework.

We also have lots of choices for PAKE schemes from post-quantum assumption, like those in [8, 11, 30, 32]. These PAKE schemes securely emulate $\mathcal{F}_{\text{PAKE}}$.

Different choices of full DH-type PAKEs and UC-secure PAKE via parallel composition or serial composition result in different hybrid PAKE (HPAKE) schemes reflecting different features. We list a few of them, whose UC security can either rely on traditional assumptions or post-quantum assumptions.

1. Round-Optimal HPAKE. Composing SPAKE2 and CrsX-GA-PAKE in parallel results in a *round-optimal* HPAKE from assumptions over groups or isogenies.

2. Efficient HPAKE. Composing SPAKE2 and the recent EKE-based PAKE scheme CHIC [8] in serial results in an *efficient* three-round HPAKE scheme based on assumptions over groups or lattices.

3. HPAKE in QROM. Composing SPAKE2 and PAKE^{QRO} [30] in serial results in a four-round HPAKE scheme, and it has *UC security in QROM* as long as the underlying PAKE^{QRO} is.

As pointed out in [23], if PAKE_2 has an extra property called “statistical PSK equality hiding”, then HPAKE from sequential composition emulates the function $\mathcal{F}_{\text{PAKE}}$ even if the party inputs incorrect passwords. The CHIC [8] has this property according to [23]. So the HPAKE scheme in **2** has no leakage on whether a party inputs a wrong password. However, PAKE^{QRO} [30] has no such property and the HPAKE scheme in **3** has 1-bit leakage when allowing parties inputting wrong passwords.

Like PAKE, our HPAKE schemes can be applied to several scenarios such as device pairing, E2E secure communication, and blockchains. The hybrid property of our HPAKE makes post-quantum migration easier for password-based cryptographic systems.

Acknowledgements. We would like to thank the reviewers for their valuable comments. This work was supported by National Key R&D Program of China under Grant 2022YFB2701500, National Cryptologic Science Fund of China 2025NCSF01009, National Natural Science Foundation of China under Grant 61925207, and Guangdong Major Project of Basic and Applied Basic Research (2019B030302008).

References

1. Anssi views on the post-quantum cryptography transition (2022). <https://cyber.gouv.fr/en/publications/anssi-views-post-quantum-cryptography-transition>
2. Cryptographic algorithms and key lengths (2024). https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.pdf?__blob=publicationFile
3. Abdalla, M., Barbosa, M., Bradley, T., Jarecki, S., Katz, J., Xu, J.: Universally composable relaxed password authenticated key exchange. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020. LNCS, vol. 12170, pp. 278–307. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-56784-2_10
4. Abdalla, M., Eisenhofer, T., Kiltz, E., Kunzweiler, S., Riepel, D.: Password-authenticated key exchange from group actions. In: Dodis, Y., Shrimpton, T. (eds.) CRYPTO 2022, Part II. LNCS, vol. 13508, pp. 699–728. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-15979-4_24
5. Abdalla, M., Haase, B., Hesse, J.: Security analysis of CPace. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021. LNCS, vol. 13093, pp. 711–741. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-92068-5_24
6. Abdalla, M., Haase, B., Hesse, J.: CPace, a balanced composable PAKE. Internet-Draft draft-irtf-cfrg-cpace-12, Internet Engineering Task Force (2024). <https://datatracker.ietf.org/doc/draft-irtf-cfrg-cpace/12/>, work in Progress
7. Abdalla, M., Pointcheval, D.: Simple password-based encrypted key exchange protocols. In: Menezes, A. (ed.) CT-RSA 2005. LNCS, vol. 3376, pp. 191–208. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-30574-3_14
8. Arriaga, A., Barbosa, M., Jarecki, S., Skrobot, M.: C’est très CHIC: A compact password-authenticated key exchange from lattice-based KEM. In: Chung, K., Sasaki, Y. (eds.) ASIACRYPT 2024, Part V. LNCS, vol. 15488, pp. 3–33. Springer (2024). https://doi.org/10.1007/978-981-96-0935-2_1
9. Barbosa, M., et al.: X-wing. CiC 1(1), 21 (2024). <https://doi.org/10.62056/a3qj89n4e>
10. Barbosa, M., Gellert, K., Hesse, J., Jarecki, S.: Bare PAKE: Universally composable key exchange from just passwords. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part II. LNCS, vol. 14921, pp. 183–217. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-68379-4_6
11. Beguinet, H., Chevalier, C., Pointcheval, D., Ricosset, T., Rossi, M.: GeT a CAKE: Generic transformations from key encapsulation mechanisms to password authenticated key exchanges. In: Tibouchi, M., Wang, X. (eds.) ACNS 23International Conference on Applied Cryptography and Network Security, Part II. LNCS, vol.

- 13906, pp. 516–538. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-33491-7_19
12. Bellare, M., Pointcheval, D., Rogaway, P.: Authenticated key exchange secure against dictionary attacks. In: Preneel, B. (ed.) EUROCRYPT 2000. LNCS, vol. 1807, pp. 139–155. Springer, Heidelberg (2000). https://doi.org/10.1007/3-540-45539-6_11
13. Bellare, S.M., Merritt, M.: Encrypted key exchange: Password-based protocols secure against dictionary attacks. In: 1992 IEEE Symposium on Security and Privacy. pp. 72–84. IEEE Computer Society Press (1992). <https://doi.org/10.1109/RISP.1992.213269>
14. Bindel, N., Hale, B.: A note on hybrid signature schemes. Cryptology ePrint Archive, Report 2023/423 (2023). <https://eprint.iacr.org/2023/423>
15. Bindel, N., Hamburg, M., Hövelmanns, K., Hülsing, A., Persichetti, E.: Tighter proofs of CCA security in the quantum random oracle model. In: Hofheinz, D., Rosen, A. (eds.) TCC 2019. LNCS, vol. 11892, pp. 61–90. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-36033-7_3
16. Bindel, N., Herath, U., McKague, M., Stebila, D.: Transitioning to a quantum-resistant public key infrastructure. In: Lange, T., Takagi, T. (eds.) PQCrypto 2017. LNCS, vol. 10346, pp. 384–405. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-59879-6_22
17. Canetti, R., Halevi, S., Katz, J., Lindell, Y., MacKenzie, P.: Universally composable password-based key exchange. In: Cramer, R. (ed.) EUROCRYPT 2005. LNCS, vol. 3494, pp. 404–421. Springer, Heidelberg (2005). https://doi.org/10.1007/11426639_24
18. Canetti, R., Rabin, T.: Universal composition with joint state. In: Boneh, D. (ed.) CRYPTO 2003. LNCS, vol. 2729, pp. 265–281. Springer, Heidelberg (2003). https://doi.org/10.1007/978-3-540-45146-4_16
19. Crockett, E., Paquin, C., Stebila, D.: Prototyping post-quantum and hybrid key exchange and authentication in TLS and SSH. Cryptology ePrint Archive, Report 2019/858 (2019). <https://eprint.iacr.org/2019/858>
20. Don, J., Fehr, S., Majenz, C., Schaffner, C.: Online-extractability in the quantum random-oracle model. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part III. LNCS, vol. 13277, pp. 677–706. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-07082-2_24
21. Giacon, F., Heuer, F., Poettering, B.: KEM combiners. In: Abdalla, M., Dahab, R. (eds.) PKC 2018. LNCS, vol. 10769, pp. 190–218. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-76578-5_7
22. Haase, B., Labrique, B.: AuCPace: efficient verifier-based PAKE protocol tailored for the IIoT. IACR TCHES **2019**(2), 1–48 (2019). <https://doi.org/10.13154/tches.v2019.i2.1-48>, <https://tches.iacr.org/index.php/TCHES/article/view/7384>
23. Hesse, J., Rosenberg, M.: PAKE combiners and efficient post-quantum instantiations. Cryptology ePrint Archive, Paper 2024/1621 (2024). <https://eprint.iacr.org/2024/1621>
24. Ishibashi, R., Yoneyama, K.: Compact password authenticated key exchange from group actions. In: Simpson, L., Baee, M.A.R. (eds.) ACISP 23. LNCS, vol. 13915, pp. 220–247. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-35486-1_11
25. Jablon, D.P.: Strong password-only authenticated key exchange. Comput. Commun. Rev. **26**(5), 5–26 (1996) <https://doi.org/10.1145/242896.242897>

26. Katz, J., Rosenberg, M.: LATKE: a framework for constructing identity-binding PAKEs. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part II. LNCS, vol. 14921, pp. 218–250. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-68379-4_7
27. Katz, J., Vaikuntanathan, V.: Smooth projective hashing and password-based authenticated key exchange from lattices. In: Matsui, M. (ed.) ASIACRYPT 2009. LNCS, vol. 5912, pp. 636–652. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-10366-7_37
28. Liu, X., Liu, S., Han, S., Gu, D.: EKE meets tight security in the Universally Composable framework. In: Boldyreva, A., Kolesnikov, V. (eds.) PKC 2023, Part I. LNCS, vol. 13940, pp. 685–713. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-31368-4_24
29. Lyu, Y., Liu, S.: Hybrid password authentication key exchange in the UC framework. Cryptology ePrint Archive, Paper 2024/1630 (2024). <https://eprint.iacr.org/2024/1630>
30. Lyu, Y., Liu, S., Han, S.: Universal composable password authenticated key exchange for the post-quantum world. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part VII. LNCS, vol. 14657, pp. 120–150. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-58754-2_5
31. Pointcheval, D., Wang, G.: VTBPEKE: Verifier-based two-basis password exponential key exchange. In: Karri, R., Sinanoglu, O., Sadeghi, A.R., Yi, X. (eds.) ASIACCS 17, pp. 301–312. ACM Press (2017). <https://doi.org/10.1145/3052973.3053026>
32. Santos, B.F.D., Gu, Y., Jarecki, S.: Randomized half-ideal cipher on groups with applications to UC (a)PAKE. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part V. LNCS, vol. 14008, pp. 128–156. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-30589-4_5
33. Shoup, V.: Security analysis of *SPAKE2+*. In: Pass, R., Pietrzak, K. (eds.) TCC 2020. LNCS, vol. 12552, pp. 31–60. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-64381-2_2